

It's the end of the term, make a friend!



PASS EXAM

期末
必过

软件工程导论



课本介绍

目录

第1章：软件工程学概述

面向结构

第2章：可行性研究

第3章：需求分析

第4章：形式化说明技术

第5章：总体设计

第6章：详细设计

第7章：实现

第8章：维护

面向对象

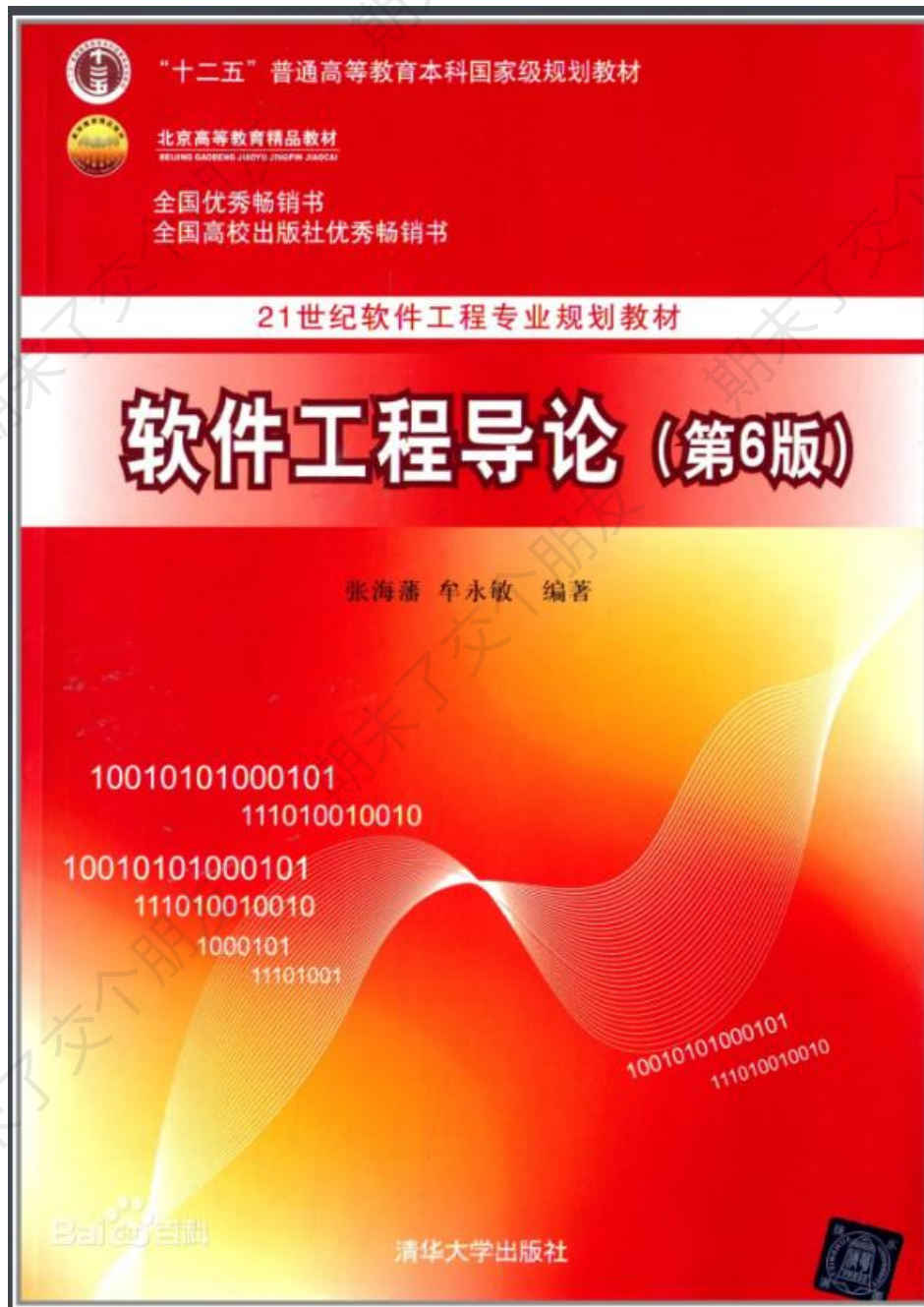
第9章：面向对象方法学引论

第10章：面向对象分析

第11章：面向对象设计

第12章：面向对象实现

第13章：软件项目管理



It's the end of the term, make a friend!



PASS EXAM

期末
必过

第一章 软件工程学概
述



一、软件危机的定义

在计算机软件开发和维护过程中所遇到的一系列严重问题。

软件危机包含两方面内容：

- 1、如何开发软件，以满足对软件日益增长的需求
- 2、如何维护数量不断膨胀的已有软件

二、软件危机的表现形式

- 对软件开发的成本和进度的估计常常不准确
- 用户对“已完成”软件系统不满意的现象经常发生
- 软件产品的质量往往靠不住
- 软件常常是不可维护的
- 软件通常没有适当的文档资料
- 软件成本在计算机系统总成本中所占比例逐年上升
- 软件开发生产率提高的速度，远远跟不上计算机应用迅速普及深入的趋势

三、软件危机的产生原因

- (1) 软件是计算机系统逻辑部件而不是物理部件，缺乏可见性，管理和控制软件相当困难
- (2) 软件显著特点是规模庞大
- (3) 存在与软件开发和维护有关的许多错误认识和做法，对用户要求没有完整准确的认识就匆忙着手编写程序
- (4) 供求矛盾、技术发展不能适应日益增长的社会需求

四、软件危机消除途径

- (1)应该对计算机软件有一个正确的认识，软件是程序，数据及相关文档的完整集合
- (2)必须充分认识到软件开发不是某种个体劳动的神秘技巧，而应该是一种组织良好、管理严密、各类人员协同配合、共同完成的工程项目
- (3)应该推广使用在实践中总结出来的开发软件的成功的技术和方法，并且研究探索更好更有效的技术和方法
- (4)应该开发和使用更好的软件工具

为了解决软件危机，既要有技术措施(方法和工具)，又要有必要的组织管理措施

一、软件工程的定义

1. 把系统的、规范的、可度量的途径应用于软件开发、运行和维护过程，也就是把工程应用于软件；
2. 研究 1 中提到的途径。

二、软件工程的本质特性

- 软件工程关注于大型软件程序的构造
- 软件工程的中心课题是控制复杂性
- 软件经常变化
- 开发软件的效率非常重要
- 和谐地合作是开发软件的关键
- 软件必须有效地支持它的用户
- 在软件工程领域中通常由具有一种文化背景的人替具有另外一种文化背景的人创造产品

三、软件工程的基本原理

- 用分阶段的生命周期计划严格管理
- 坚持进行阶段评审
- 实行严格的产品控制
- 采用现代程序设计技术
- 结果应能清楚地审查
- 开发小组的人员应该少而精
- 承认不断改进软件工程实践的必要性

这7条原理是确保软件产品质量和开发效率的原理的最小集合，它们是互相独立的，其中任意6条原理的组合都不能代替另一条原理

四、软件工程方法学

- (1) 把在软件生命周期全过程中使用的一整套技术方法的集合称为方法学,也称为泛型。
- (2) 软件工程方法学包括三个要素：**方法**、**工具**和**过程**
- (3)
 - a. 方法是完成软件开发的各项任务的技术方法，回答“怎样做”的问题；
 - b. 工具是为运用方法而提供的自动的或半自动的软件工程支撑环境；
 - c. 过程是为了获得高质量的软件所需要完成的一系列任务的框架，它规定了完成各项任务的工作步骤。

五、软件工程方法学分类

1. 传统方法学(结构化范型)

- (1) 采用结构化技术(结构化分析、结构化设计和结构化实现)完成软件开发的各项任务。
- (2) 把软件生命周期划分成若干个阶段,然后顺序完成各个阶段的任务。
- (3) 每个阶段的开始和结束都有严格的标准,对于任何两个相邻的阶段而言,前一阶段的结束标准就是后一阶段的开始标准。
- (4) 在每个阶段结束之前都必须正式地进行严格的技术审查和管理复审。

五、软件工程方法学分类

2. 面向对象方法学(面向对象范型)

(1) 把对象作为融合了数据及在数据上操作的软件构件。即用对象分解取代了传统方法的功能分解。

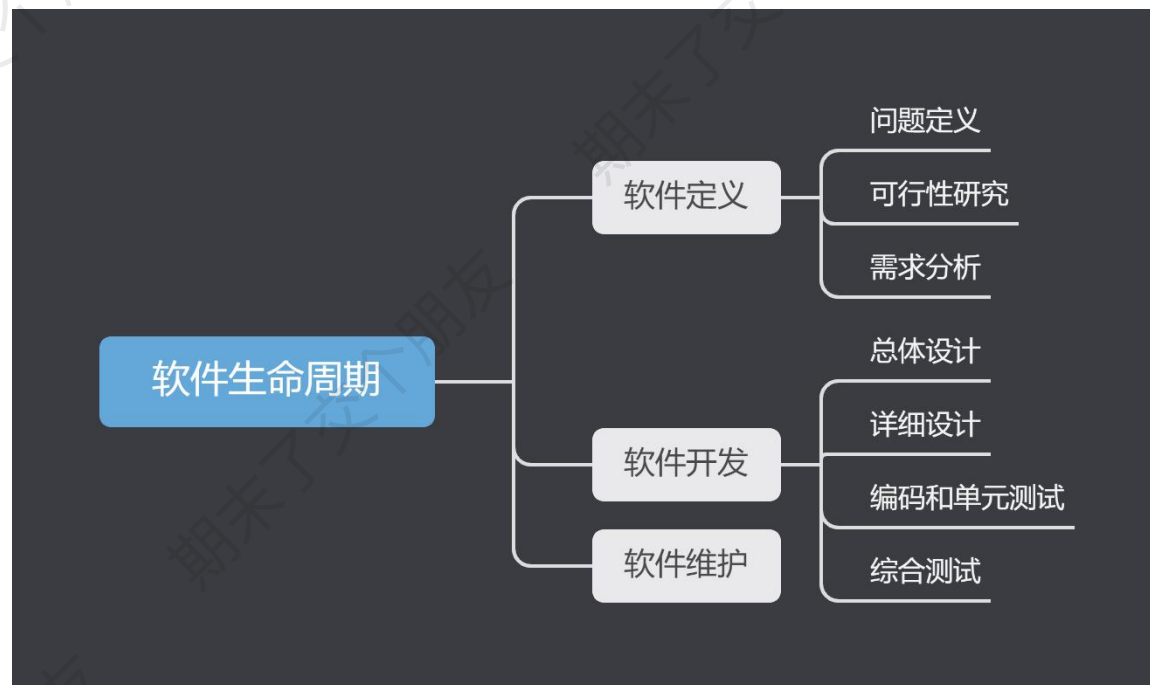
(2) 把所有对象都划分成类。

(3) 按照父类与子类的关系,把若干个相关类组织成一个层次结构的系统。

(4) 对象彼此间仅能通过发送消息互相联系。

(1)软件生命周期由软件定义、软件开发和运行维护(也称为软件维护)3 个时期组成，每个时期又进一步划分成若干个阶段。

(2)软件定义时期划分为 3 个阶段即问题定义、可行性研究和需求分析；软件开发时期划分为 4 个阶段即总体设计、详细设计、编码和单元测试、综合测试以及软件维护时期。



软件过程:是为了获得高质量软件所需要完成的一系列任务框架。
通常用软件生命周期模型描述软件过程。

主要包括:

- 瀑布模型
- 快速原型模型
- 增量模型
- 螺旋模型
- 喷泉模型
- 其他模型

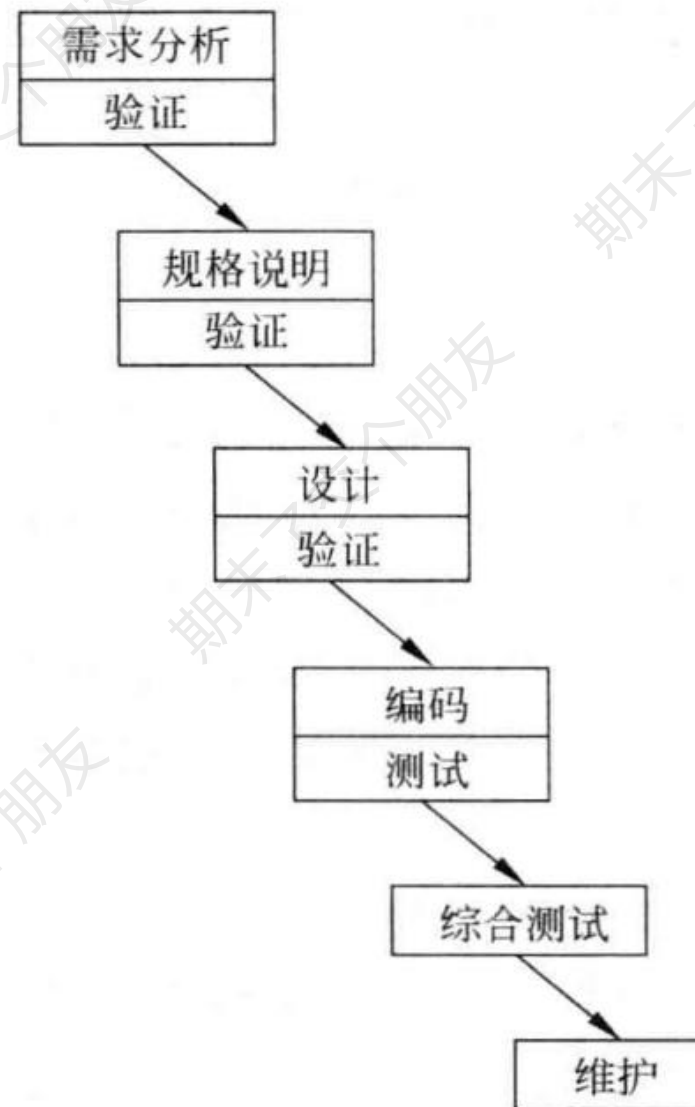
一、瀑布模型(应用最广泛)

优点：

- a. 可强迫开发人员采用规范方法
- b. 严格规定了每个阶段必须提交的文档
- c. 要求每阶段交出所有产品都必须经过质量保证小组的仔细检验

缺点：

- a. 只能通过文档了解产品，不经过实践的需求是不切实际的
- b. 需求确定一段时间之后才能获得产品的最初版本(软件实际情况必须到项目开发后期客户才能看到)
- c. 完全依赖于规格说明导致不能满足用户需求；
- d. 客户往往很难清楚地给出所有需求。



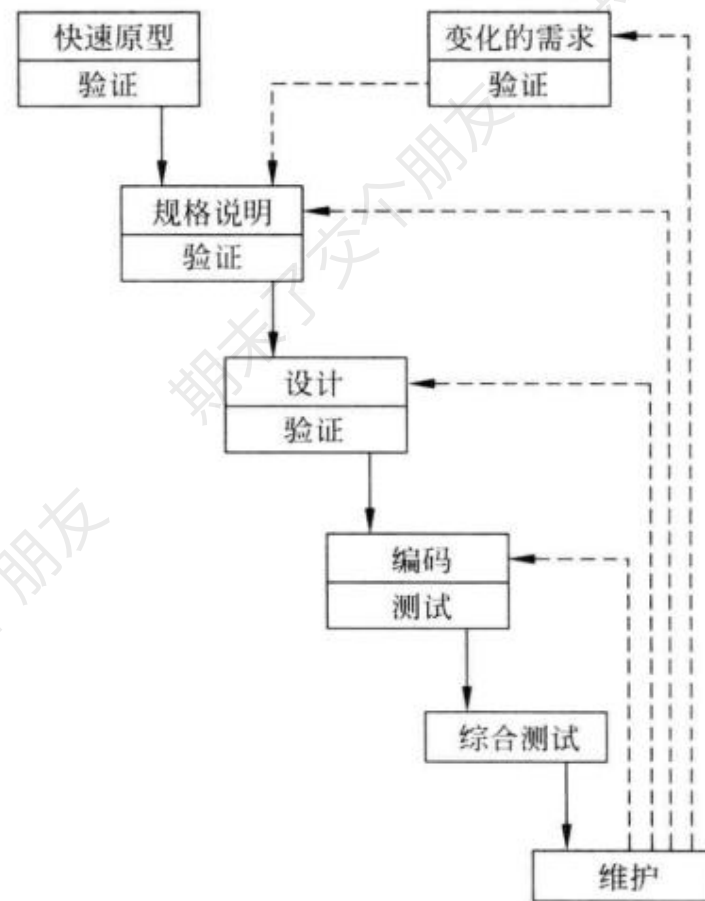
二、快速原型模型

优点：

- a.使用这种软件过程开发的软件产品通常能满足用户的真实需求
- b.软件产品开发过程基本是线性顺序的
- c.节省开发的投入，缩短整个软件开发周期
- d.改善用户参与
- e.提高系统的实用性和可维护性

缺点：

- a.用户和开发者对原型理解不同，用户有时误解原型角色，误以为其与真实系统一样可靠
- b.准确的原型设计比较困难
- c.由于用户不断提出新要求，原型迭代周期很难控制
- d.为了尽快实现原型，采用不合适的技术会使运行效率受到影响



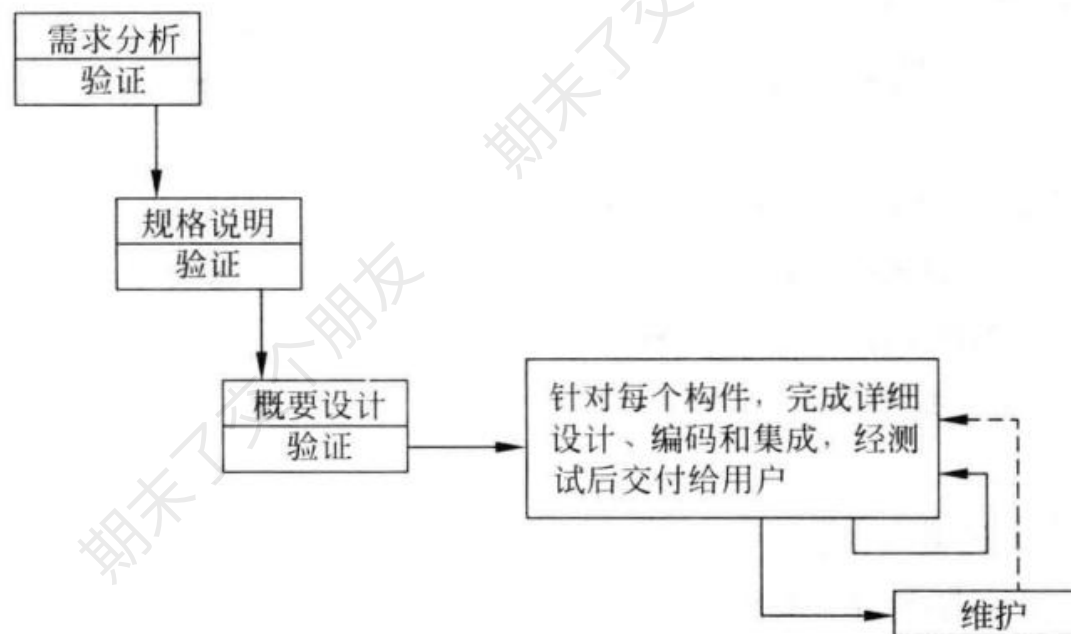
三、增量模型

优点：

- a. 能在较短时间内向用户提交可完成部分工作产品
- b. 逐步增加产品功能，从而使用户有充裕的时间学习和适应新产品，减少一个全新软件给用户带来的冲击
- c. 开放式软件可维护性比较好
- d. 在第一次构建完成之前就已经完成总体设计，风险较小

缺点：

- a. 为成功开发软件，软件工程师必须具有较高水平技术，能设计出开放的软件体系结构
- b. 把每一个新的增量构件集成到现有的软件体系结构中必须破坏原来的已经开发出的产品，使软件过程的控制失去整体性



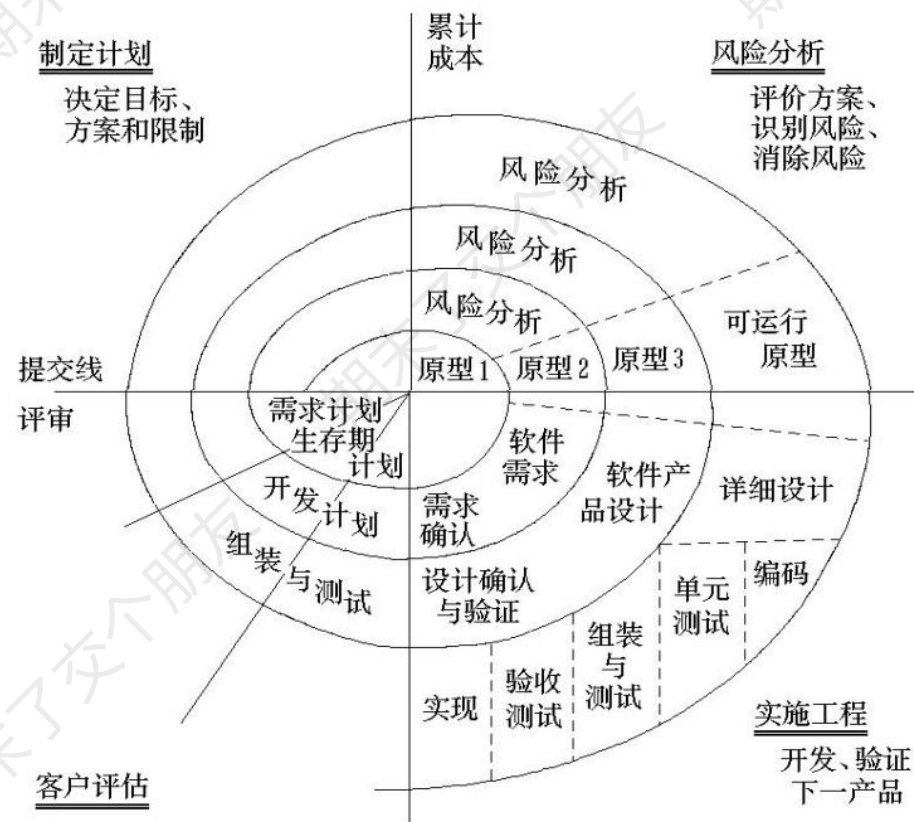
(1)螺旋模型的基本思想是使用原型及其他方法来尽量降低风险，可以把它看作在每个阶段之前都增加了风险分析过程的快速原型模型

(2) 优点：

- a.有利于已有软件的重用
- b.有助于把软件质量作为软件开发的一个重要目标
- c.减少了过多测试或测试不足所带来的风险
- d.软件维护与软件开发没有本质区别

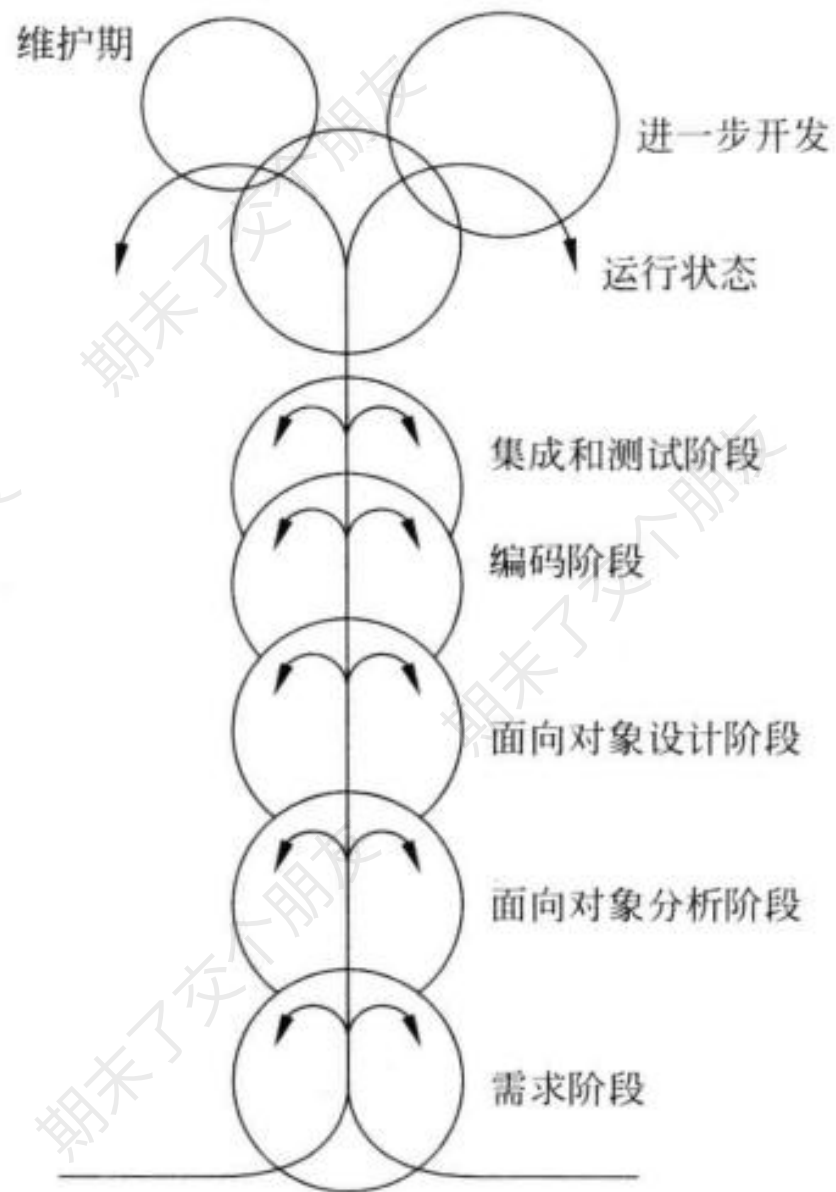
(3)缺点：

使用螺旋模型开发软件，要求软件开发人员具有丰富的风险评估知识和经验

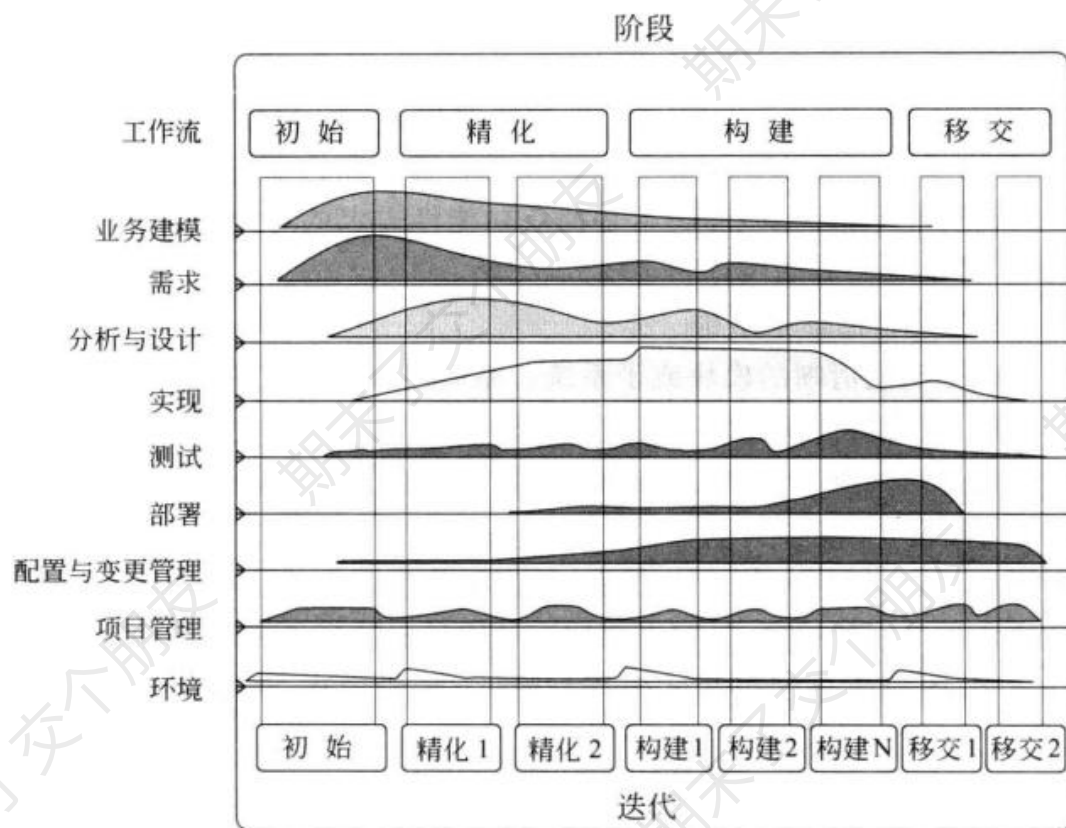


五、喷泉模型(面向对象的模型)

迭代和无缝的特性



六、RUP模型



(1)最佳实践

- a.迭代式开发
- b.管理需求
- c.使用基于构件的体系结构
- d.可视化建模
- e.验证软件质量
- f.控制软件变更

(2)9个核心 workflow

业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理和环境(前 6 个为核心过程 workflow,后 3 个为核心支持 workflow)

(3)工作阶段

初始阶段、精化阶段、构建阶段和移交阶段

七、敏捷过程(包含极限编程)

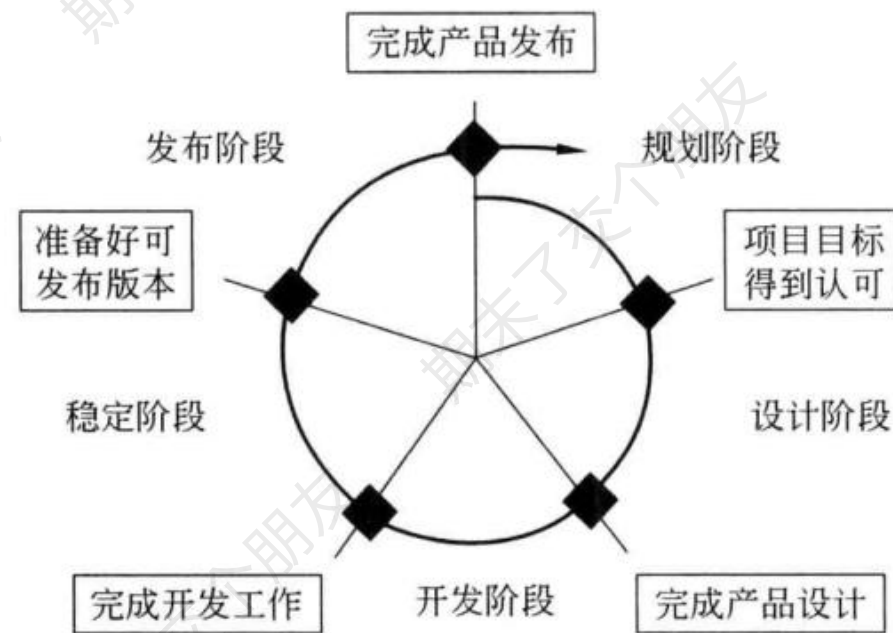
(1)敏捷开发宣言

- a.个体和交互胜过过程和工具
- b.可以工作的软件胜过面面俱到的文档
- c.客户合作胜过合同谈判
- d.响应变化胜过遵循计划

(2)应用场景：

能够较好地适应**商业竞争环境**下对小型项目提出的**有限资源**和**有限开发时间**的约束

八、微软过程



例题1 软件工程的出现主要是由于 ()

A.程序设计方法学的影响

B.其它工程科学的影响

C.软件危机的出现

D.计算机的发展

例题2 适合于面向对象方法学的软件生存周期模型是 ()

A.瀑布模型

B.喷泉模型

C.螺旋模型

D.增量模型

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第二章 可行性研究



可行性研究的任务

(1) 可行性研究的目的是不是解决问题,而是确定问题是否值得去解决

(2) 可行性

a. 技术可行性

b. 经济可行性

c. 操作可行性

可行性研究过程

- 复查系统规模和目标
- 研究目前正在使用的系统
- 导出新系统的高层逻辑模型
- 进一步定义问题
- 导出和评价选择的解法
- 推荐行动方针
- 草拟开发计划
- 书写文档提交审查

系统流程图是概括地描绘物理系统的传统工具。它的基本思想是用图形符号以黑盒子形式描绘组成系统的每个部件(程序、文档、数据库、人工过程等)。系统流程图表达的是数据在系统各部件之间流动的情况，而不是对数据进行加工处理的控制过程。

符 号	名 称	说 明
	处理	能改变数据值或数据位置的加工或部件，例如程序、处理机、人工加工等都是处理
	输入输出	表示输入或输出（或既输入又输出），是一个广义的不指明具体设备的符号
	连接	指出转到图的另一部分或从图的另一部分转来，通常在同一页上
	换页连接	指出转到另一页图上或由另一页图转来
	数据流	用来连接其他符号，指明数据流动方向

(1)定义：数据流图(DFD)是一种图形化技术，它描述信息流和数据流从输入移动到输出的过程中所经受的变换。在数据流图中没有任何具体的物理部件，它只是描绘数据在软件中流动和被处理的逻辑过程。

(2)数据流图有4种基本符号：

- a.正方形 / 立方体表示数据的源点或终点
- b.圆角矩形 / 圆形表示数据的处理
- c.开口矩阵 / 两条平行横线表示数据存储
- d.箭头表示数据流



(1)定义：数据字典是关于数据的信息的集合，也就是对数据流图中包含的所有元素的定义的集合。

(2)数据字典的 4 类元素定义组成：数据流、数据流分量 / 数据元素、数据存储和处理

(3)关系：数据流图和数据字典构成系统的逻辑模型，没有数据字典，数据流图就不严格，然而没有数据流图，数据字典也能难以发挥作用。只有数据流图和数据字典中每个元素的精确定义放在一起，才能共同构成系统的规格说明

(1)=：定义于/等价于

(2)+：与/和(连接两个分量)

(3)[... | ...]：或(从中括号内列出的若干个分量中选择一个)

(4){...}：重复(重复大括号内的分量)

(5) $m\{\dots\}n$ 或 ${}^m_n\{\dots\}$ ：重复(使用上限和下限进一步注释表示重复的次数，其中一种注释方法 ${}^m_n\{\dots\}$ 在开括弧的左边用上角标和下角标分别表明重复的上限和下限；另一种注释方法 $m\{\dots\}n$ 是在开括弧左侧标明重复的下限,在闭括弧的右侧标明重复的上限)

(6)“...”：连接符。例如， $x=1\cdots 9$ ，表示 x 可取1到9中任意一个值。

(1)成本效益分析的目的正是要从经济角度分析开发一个特定的新系统是否划算，从而帮助客户组织的负责人正确地做出是否投资于这项开发工程的决定

(2)成本估计的方法：

- a. 代码行技术
- b. 任务分解技术
- c. 自动估计成本技术

(3)成本效益分析的指标：

- a. 货币的时间价值($P=F/(1+i)^n$ ，即若n年后可以收入F元钱，则这些钱现在的价值P)
- b. 投资回收期
- c. 纯收入
- d. 投资回收率

例题1 软件工程的出现主要是由于 ()

- A.程序设计方法学的影响
- B.其它工程科学的影响
- C.软件危机的出现
- D.计算机的发展

例题2 适合于面向对象方法学的软件生存周期模型是 ()

- A.瀑布模型
- B.喷泉模型
- C.螺旋模型
- D.增量模型

- 例题3** 请根据以下描述画出实例考务处理系统的DFD图（包括顶层、0层和1层数据流图）。
- (1) 对考生送来的报名单进行检查;
 - (2) 对合格的报名单编好准考证号将准考证送给考生，并将汇总后的考生名单送给阅卷站;
 - (3) 对阅卷站送来的成绩单进行检查，并根据考试中心制定的合格标准审定合格者;
 - (4) 制作考生通知单（含成绩合格/不合格标志）送给考生;
 - (5) 按地区进行成绩分类统计和试题难度分析，产生统计分析表。

例题4 北京某高校可用的电话号码有以下几类：校内电话号码由4位数字组成，第一位数字不是0。校外电话又分为本市电话和外地电话两类。拨校外电话需要先拨0，若是本市电话则接着拨8位数字(第一位不是0)，若是外地电话则拨3位区码后再拨8位电话号码(第一位不是0)。

电话号码 = [校内电话 | 校外电话]

校内电话 = 非零数字 + 三位数字

非零数字 = [1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9]

三位数字 = 3 {数字} 3

数字 = [0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9]

校外电话 = [本市电话 | 外地电话]

本市电话 = 0 + 八位非零开头数字

八位非零开头数字 = 非零数字 + 七位数字

七位数字 = 7 {数字} 7

外地电话 = 0 + 三位区码 + 八位非零开头数字

三位区码 = 三位数字

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第三章 需求分
析



一、结构化需求分析方法遵守准则

- (1)必须理解并描述问题的信息域，根据这条准则应该建立数据模型；
- (2)必须定义软件应完成的功能，这条规则要求建立功能模型；
- (3)必须描述作为外部事件结果的软件行为，这条准则要求建立行为模型；
- (4)必须对描述信息、功能和行为的模型进行分解，用层次的方式展示细节。

二、需求分析任务

- a. 确定对系统的综合要求(功能需求、性能需求、可靠性和可用性需求、出错处理需求、接口需求、约束、逆向需求和将来可能提出的要求)
- b. 分析系统的数据要求
- c. 导出系统逻辑模型
- d. 修正系统开发计划

与用户沟通获取需求的方法

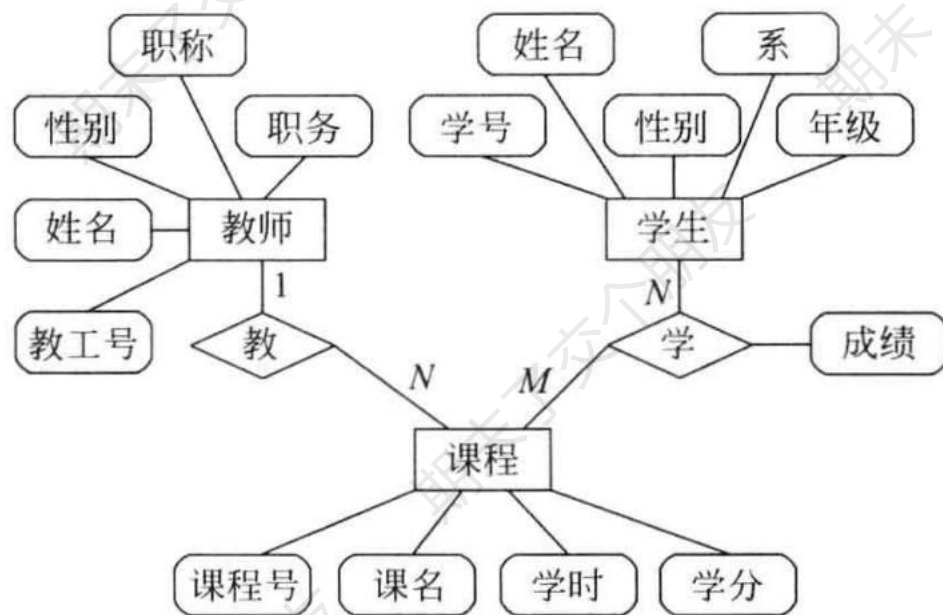
- (1) 访谈(正式和非正式)
- (2) 面向数据流自顶向下求精
- (3) 简易的应用规格说明技术(面向团队的需求收集方法)
- (4) 快速建立软件原型(快速原型两个特性：**快速和容易修改**)(快速构建和修改原型方法和工具：第四代技术，可重用软件构件，形式化规格说明和原型环境)

(1)数据模型--实体-联系图

(2)功能模型--数据流图

(3)行为模型--状态转换图

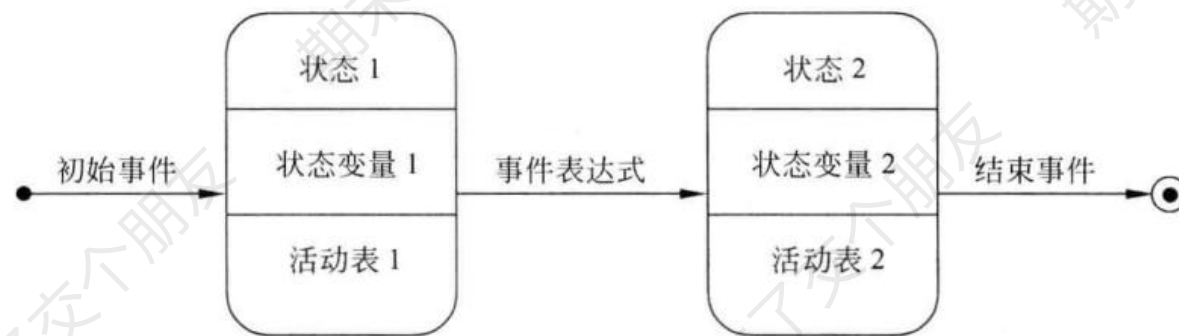
- (1) 矩形框：实体，内写实体名
- (2) 椭圆形：属性，内写属性名
- (3) 菱形：联系，内写联系名(1:1 或 1:n 或 m:n)
- (4) 直线：实体与属性之间；实体与联系之间；联系与属性之间用直线相连



(1)通常用**范式**定义消除数据冗余的程度。第一范式(1 NF)数据冗余程度最大,第五范式(5 NF)数据冗余程度最小

(2)第一/第二/第三范式

- (1)初态：实心圆
- (2)终态：一对同心圆(内圆为实心圆)
- (3)中间状态：圆角矩形
- (4)状态转换：带注释的箭头连线



(1)层次方框图用树形结构的一系列多层次的矩形框描绘数据的层次结构

(2)Warnier 图用树形结构描绘信息，和层次方框图类似，但是这种图形工具比层次方框图提供了更丰富的描绘手段

(3)IPO 图是输入、处理、输出图的简称，能够方便地描绘输入数据、对数据的处理和输出数据之间的关系

需求分析使用的图形工具

数据流图

E-R图

状态转换图（状态图）

层次方框图

Warnier图

IPO图

- (1)一致性：所以需求必须是一致的，任何一条需求不能和其他需求相互矛盾。
- (2)完整性：需求必须是完整的，规格说明书应该包括用户需要的每一个功能或性能。
- (3)现实性：指定的需求应该用现有的硬件技术和软件技术基本上可以实现的。
- (4)有效性：必须证明需求是正确有效的，确实能解决用户面对的问题。

例题1 需求分析阶段最重要的技术文档是_____。

需求规格说明书

例题2 需求分析阶段的任务是确定（ ）

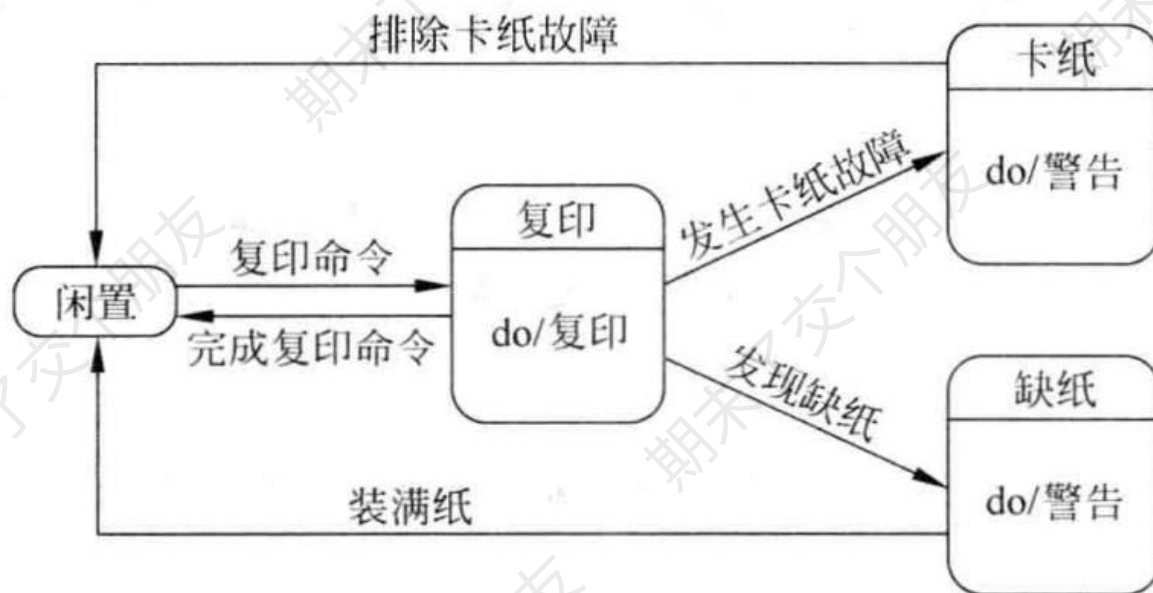
A.软件开发方法

B.软件开发工具

C.软件开发费

D.软件系统的功能

例题3 复印机的工作过程大致如下:未接到复印命令时处于闲置状态,一旦接到复印命令则进入复印状态,完成一个复印命令规定的工作后又回到闲置状态,等待下一个复印命令;如果执行复印命令时发现没纸,则进入缺纸状态,发出警告,等待装纸,装满纸后进入闲置状态,准备接收复印命令;如果复印时发生卡纸故障,则进入卡纸状态,发出警告,等待维修人员来排除故障,故障排除后回到闲置状态。试用状态转换图描绘复印机的行为。



It's the end of the term, make a friend!



PASS EXAM

期末
必过

第四章
术

形式化说明技



(1)按照形式化的程度,可以把软件工程使用的方法划分成非形式化、半形式化和形式化3类。用自然语言描述需求规格说明,是典型的非形式化方法。用数据流图或实体-联系图建立模型,是典型的半形式化方法。有穷状态机、Petri 网和Z语言是典型的形式化方法。

(2)形式化方法,是描述系统性质的基于数学的技术,也就是说,如果一种方法有坚实的数学基础,那么它就是形式化的。

(3)非形式化方法的缺点：用自然语言书写的系统规格说明书,可能存在矛盾、二义性、含糊性、不完整性及抽象层次混乱等问题。

(4)形式化方法的优点：

- a.能够简洁准确地描述物理现象、对象或动作的结果,是理想的建模工具。
- b.可以在不同的软件工程活动之间平滑地过渡。
- c.它提供了高层确认的手段。

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第五章 总体设计



系统设计阶段

- (1)设计供选择的方案
- (2)选取合理的方案
- (3)推荐最佳方案

结构设计阶段

- (4)功能分解
- (5)设计软件结构
- (6)设计数据库
- (7)制定测试计划
- (8)书写文档
- (9)审查与复审

总体设计(概要设计)过程通常由两个主要阶段组成：系统设计阶段,确定系统的具体实现方案;结构设计阶段,确定软件结构

一、模块化

(1)模块：模块是由边界元素限定的相邻程序元素的序列，而且总有一个总体标识符代表它

(2)模块化：模块化就是把程序划分成独立命名且可独立访问的模块，每个模块完成一个子功能，把这些模块集成起来构成一个整体，可以完成指定的功能满足用户的需求

(3)模块化的作用/优点：

- a.可以使软件结构清晰，不仅容易设计也容易阅读和理解
- b.使软件容易测试和调试，因而有助于提高软件的可靠性
- c.模块化能够提高软件的可修改性
- d.模块化也有助于软件开发工程的组织管理

二、抽象

抽象出事物的本质特性而暂时不考虑它们的细节。

三、逐步求精

为了能集中精力解决主要问题而尽量推迟对问题细节的考虑(Miller法则：一个人在任何时候都只能把注意力集中在 (7 ± 2) 个知识块上)

四、信息隐藏与局部化

(1)信息隐藏：在设计和确定模块时，使得一个模块内包含的信息，对那些不需要这些信息的其他模块来说，是不能访问的，也是不可见的

(2)局部化：把一些关系密切的软件元素物理地放得彼此靠近，它有助于实现信息隐藏

五、模块独立

(1)模块独立的概念是模块化、抽象、信息隐藏和局部化概念的直接结果

(2)模块独立性重要的原因：

a.有效的模块化(即具有独立的模块)的软件比较容易开发出来；

b.独立的模块比较容易测试和维护

(3)度量标准：

a.内聚：衡量一个模块内部各个元素彼此结合的紧密程度。

b.耦合：衡量不同模块彼此间互相依赖的紧密程度。

五、模块独立

- (1)数据耦合：两个模块之间仅仅通过参数交换信息，而且交换的仅仅是数据
- (2)控制耦合：两个模块之间传递的信息有控制信息
- (3)特征耦合：把整个数据结构作为参数传递，而被调用的模块只需要使用其中一部分数据元素
- (4)公共环境耦合：连个或多个模块通过一个公共区相互作用
- (5)内容耦合：
有下列情形之一，两个模块间就发生了内容耦合：
 - a. 一个模块访问一个模块的内部数据
 - b. 一个模块不通过正常入口而转到另一个模块的内部
 - c. 两个模块有一部分程序代码重叠(只可能出现在汇编程序中)
 - d. 一个模块有多个入口(这意味着一个模块有几种功能)

五、模块独立

- (1)偶然内聚：如果一个模块完成一组任务，这些任务彼此间即使有关系，关系也是很松散的，就叫做偶然内聚
- (2)逻辑内聚：如果一个模块完成的任务在逻辑上属于相同或相似的一类，则称为逻辑内聚
- (3)时间内聚：如果一个模块包含的任务必须在同一段时间内执行（例如：模块完成各种初始化工作），就叫做时间内聚
- (4)过程内聚：如果一个模块内的处理元素是相关的，而且必须以特定次序执行，则称为过程内聚。
- (5)通信内聚：如果模块中所有元素都使用同一输入数据和（或）产生同一个输出数据，则称为通信内聚
- (6)顺序内聚：如果一个模块内的处理元素和同一个功能密切相关，而且这些处理必须顺序执行则称为顺序内聚
- (7)功能内聚：如果模块内所有处理元素属于一个整体，完成一个单一的功能，则称为功能内聚

五、模块独立

(1)耦合程度从低到高依次是：数据<控制<特征<公共环境<内容

(2)内聚程度从高到低依次是：功能>顺序>通信>过程>时间>逻辑>偶然



- (1)改进软件结构提高模块独立性
- (2)模块规模应该适中
- (3)深度、宽度、扇出和扇入都应适当
- (4)模块的作用域应该在控制域之内
- (5)力争降低模块接口的复杂程度
- (6)设计单入口单出口的模块
- (7)模块功能应该可以预测

深度：软件控制的层数宽度即软件结构内同一层次上的模块总数的最大值

扇出：一个模块直接控制的模块数目

扇入：表明有多少个上级模块直接调用它

作用域：受该模块内一个判定影响的所有模块集合

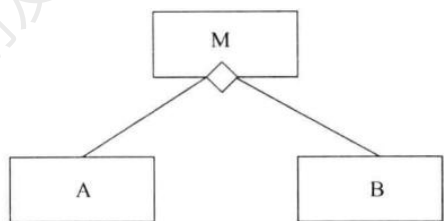
控制域：这个模块本身以及所有直接或间接从属于它的模块的集合。

描绘软件结构的图形工具

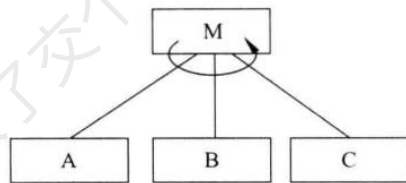
(1)模块：矩形框，框内注明模块的名字或主要功能

(2)模块间的调用关系：直线/带箭头的直线

(3)模块调用过程中传递的信息：带注释的箭头，其中注释箭头尾部为空心圆表示传递的是数据，注释箭头尾部为实心圆表示传递的是控制信息



条件调用



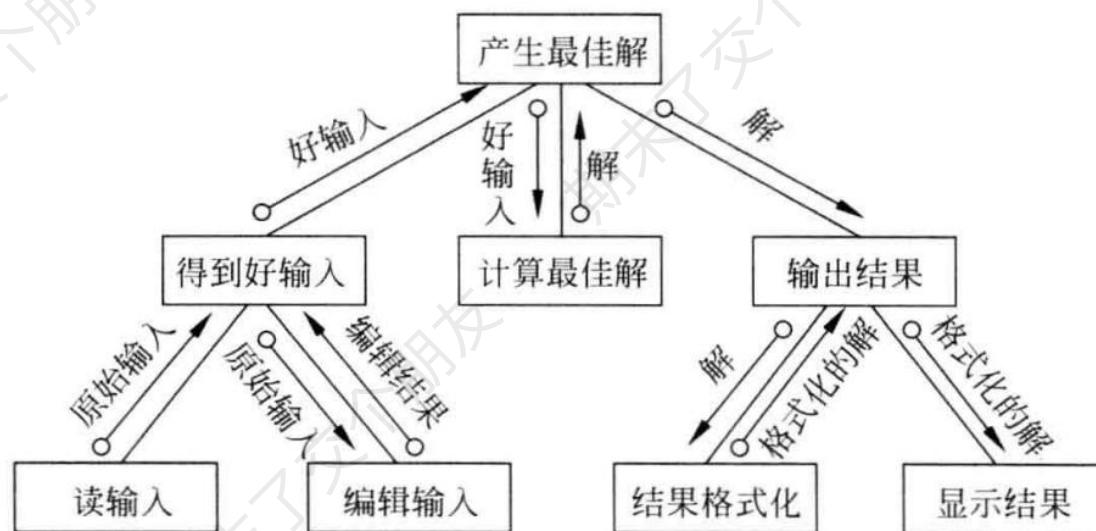
循环调用

总体设计使用/描绘软件结构的图形工具

层次图

HIPO图

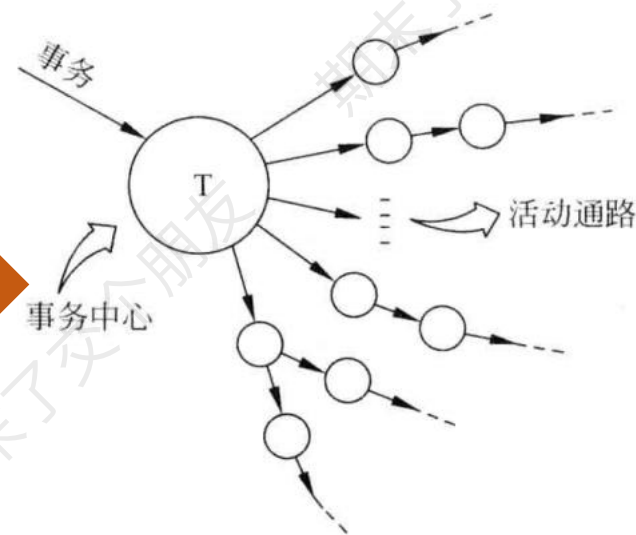
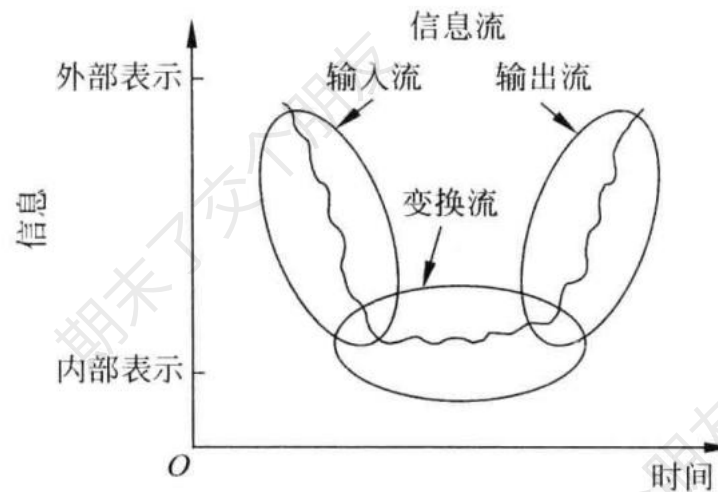
结构图



信息流的两种类型：

(1)变换流：信息沿输入通路进入系统,同时由外部形式变换成内部形式,进入系统的信息通过变换中心,经加工处理以后再沿输出通路变换成外部形式离开软件系统。当数据流图具有这些特征时,这种信息流就叫作变换流

(2)事务流：数据沿输入通路到达一个处理T,这个处理根据输入数据的类型在若干个动作序列中选出一个来执行。这类数据流应该划为一类特殊的数据流,称为事务流



例题1 在进行软件结构设计时应遵循的最主要的原理是（ ）

- A.抽象
- B.模块独立
- C.模块化
- D.信息隐藏

例题2 两个模块之间通过传递公共指针或地址相互作用的耦合为（ ）

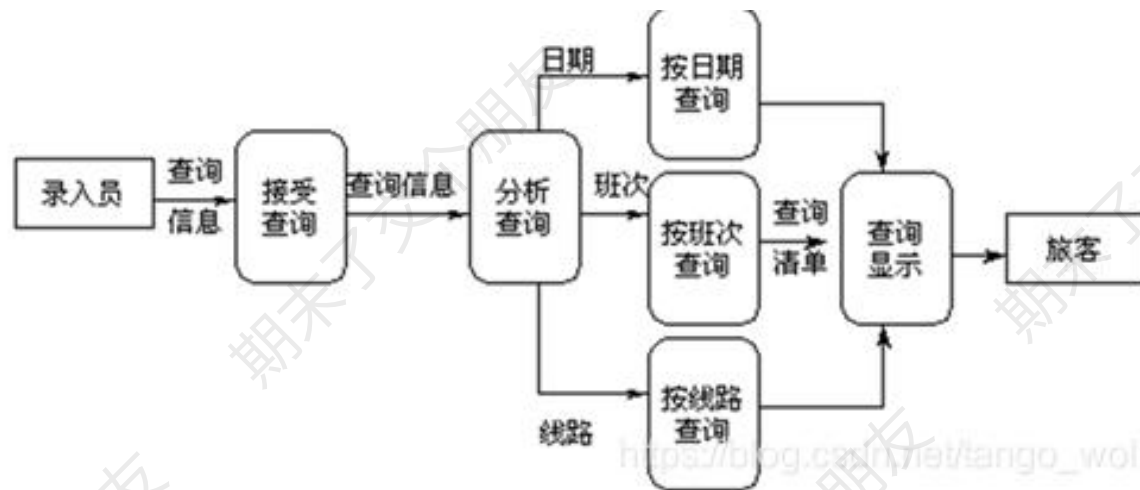
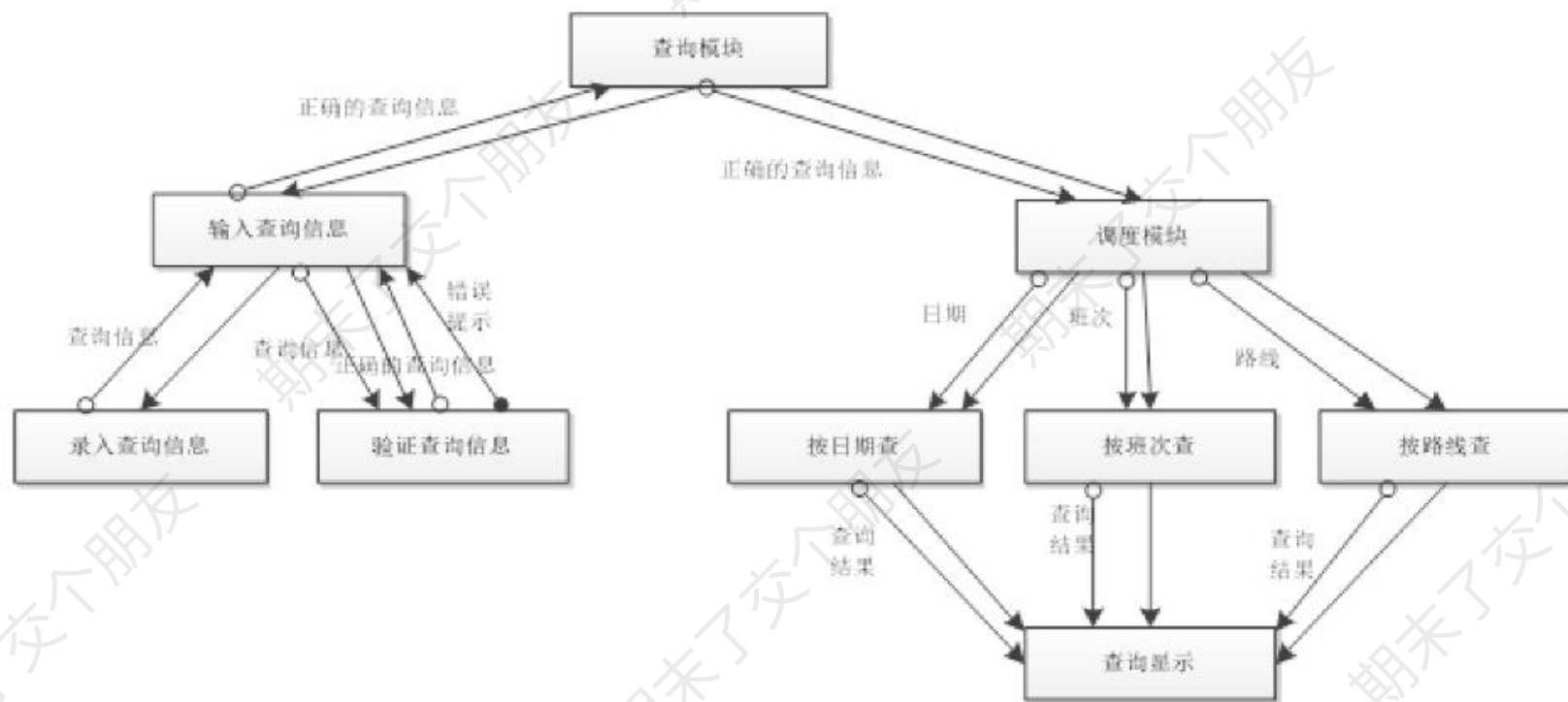
- A.公共环境耦合
- B.数据耦合
- C.标记耦合
- D.控制耦合

例题3 按照软件工程的原则，模块的作用域和模块的控制域之间的关系是（ ）

- A.模块的作用域应在模块的控制域之内
- B.模块的控制域应在模块的作用域之内
- C.模块的控制域与模块的作用域互相独立
- D.以上说法都不对

例题4 下面几种类型的耦合类型中，耦合程度最低的是（ ）

- A.控制耦合
- B.内容耦合
- C.公共耦合
- D.数据耦合

例题3 将下列数据流图转换成软件结构图。

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第六章 详细设
计



如果一个程序的代码块仅仅通过**顺序、选择和循环**这 3 种基本控制结构进行连接，并且每个代码块**只有一个入口和一个出口**，则称这个程序是结构化的

(1)设计人机界面过程遇到的4个问题

- a.系统响应时间
- b.用户帮助设施
- c.出错信息处理
- d.命令交互

(2)人机界面设计指南

1)一般交互指南

- a. 保持一致性
- b. 提供有意义的反馈
- c. 在执行有较大破坏性的动作之前要求用户确认
- d. 允许取消大多数操作
- e. 允许犯错误

2)信息显示指南

- a. 只显示与当前工作有关的信息
- b. 不要用数据淹没用户，应该用便于用户迅速吸取信息的方式来表示数据
- c. 使用一致的标记、标准的缩写和可预知的颜色
- d. 允许用户保持可视化的语境
- e. 产生有意义的出错信息

3)数据输入指南

- a. 尽量减少用户的输入动作
- b. 保持信息显示和数据输入之间的一致性
- c. 允许用户自定义输入
- d. 让用户控制交流
- e. 消除冗余的输入

(1)描述程序处理过程的工具称为过程设计的工具，它们可以分为**图形、表格和语言**

(2)对程序(特别是复杂的大型程序)的设计,通常分为两个阶段完成:首先进行结构设计,然后进行过程设计。结构设计确定程序由哪些模块组成,以及这些模块之间的关系;过程设计确定每个模块的处理过程。**结构设计是总体设计阶段的任务(总体设计又称为概要设计或结构设计),过程设计是详细设计阶段的任务(详细设计又称为模块设计或过程设计)**

详细设计使用/过程设计的工具

程序流程图

N-S图 (盒图)

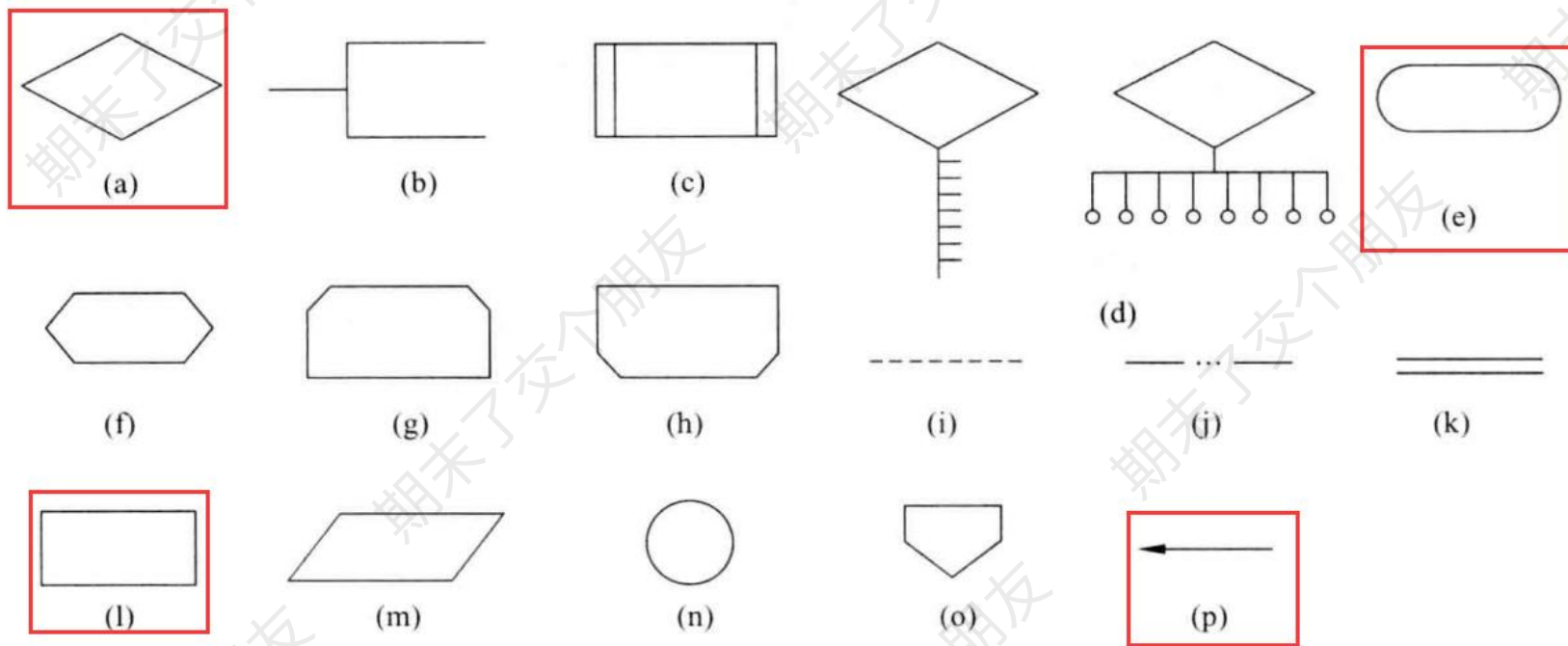
PAD图 (问题分析图)

判定表

判定树

PDL(过程设计语言/伪码)

一、程序流程图



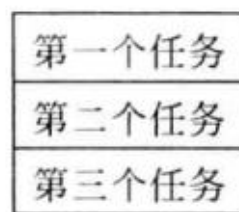
(a) 选择(分支); (b) 注释; (c) 预先定义的处理; (d) 多分支; (e) 开始或停止; (f) 准备;

(g) 循环上界限; (h) 循环下界限; (i) 虚线; (j) 省略符; (k) 并行方式; (l) 处理;

(m) 输入输出; (n) 连接; (o) 换页连接; (p) 控制流

二、盒图

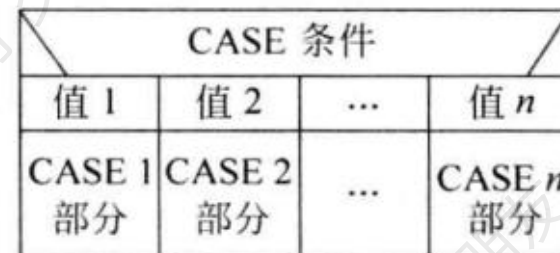
- (1) 功能域明确，可以从盒图上一眼就看出来
- (2) 不可能任意转移控制
- (3) 很容易确定局部和全程数据的作用域
- (4) 很容易表现嵌套关系，也可表示模块的层次结构



(a)



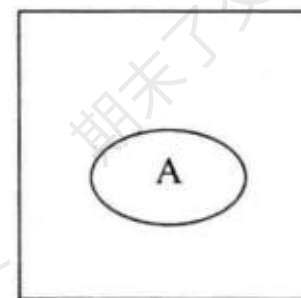
(b)



(c)



(d)

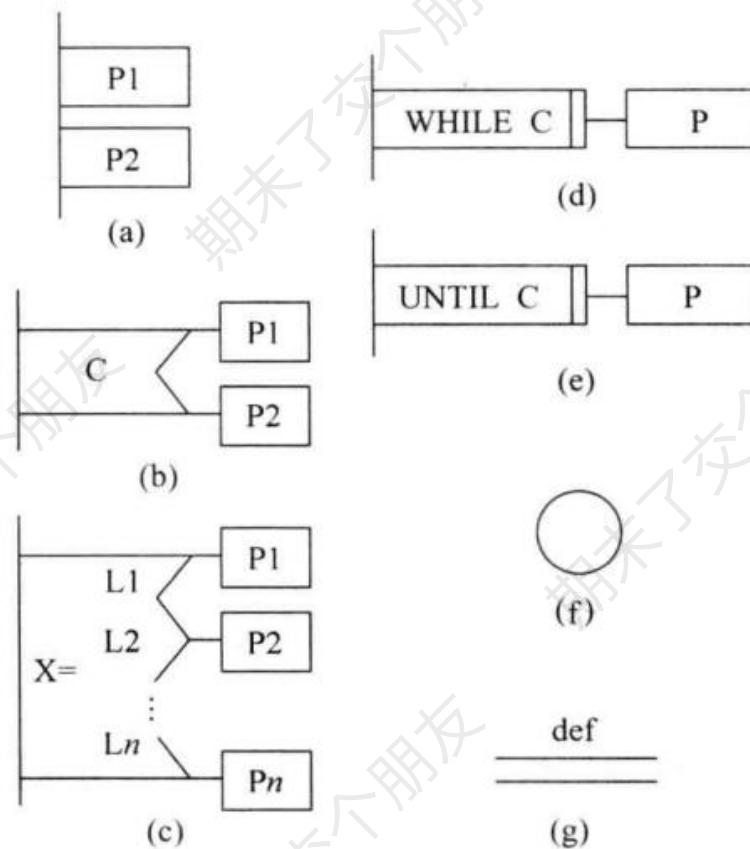


(e)

(a) 顺序；(b) IF_THEN_ELSE 型分支；(c) CASE 型多分支；(d) 循环；(e) 调用子程序 A

三、PAD图

- (1)使用表示结构化控制结构的 PAD 符号所设计出来的程序必然是结构化程序
- (2)PAD 图所描绘的程序结构十分清晰
- (3)用 PAD 图表现程序逻辑，易读、易懂、易记
- (4)容易将 PAD 图转换成高级语言源程序
- (5)即可用于表示程序逻辑，也可用于描绘数据结构
- (6)PAD 图的符号支持自顶向下、逐步求精方法的使用



(a) 顺序(先执行 P1 后执行 P2); (b) 选择(IF C THEN P1 ELSE P2); (c) CASE 型多分支;
(d) WHILE 型循环(WHILE C DO P); (e) UNTIL 型循环(REPEAT P UNTIL C); (f) 语句标号; (g) 定义

四、判定表&&判定树

(1)判定表/判定树却能够清晰地表示复杂的条件组合与应做的动作之间的对应关系

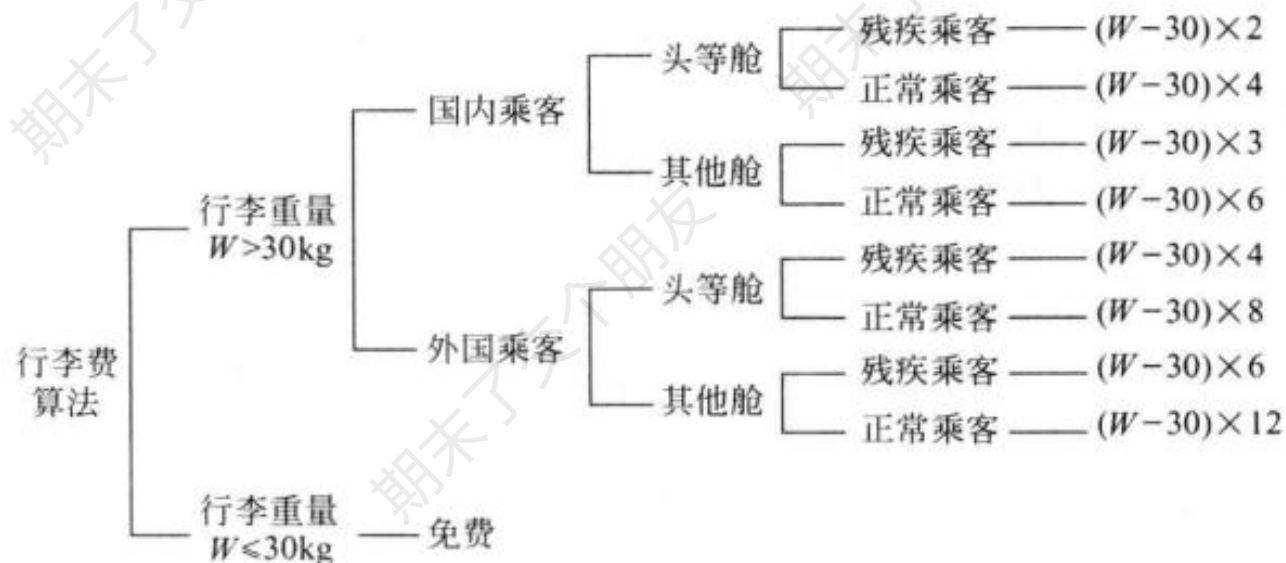
(2)一张判定表由4部分组成,左上部列出所有条件,左下部是所有可能做的动作,右上部是表示各种条件组合的一个矩阵,右下部是和每种条件组合相对应的动作。判定表右半部的每一列实质上是一条规则,规定了与特定的条件组合相对应的动作

(3)虽然判定树比判定表更直观,但简洁性却不如判定表,数据元素的同一个值往往要重复写多遍,而且越接近树的叶端重复次数越多

四、判定表&&判定树

假设某航空公司规定,乘客可以免费托运重量不超过30kg的行李。当行李重量超过30kg时,对头等舱的国内乘客超重部分每公斤收费4元,对其他舱的国内乘客超重部分每公斤收费6元,对外国乘客超重部分每公斤收费比国内乘客多一倍,对残疾乘客超重部分每公斤收费比正常乘客少一半。请用判定表和判定树表示行李托运算法。

规则									
	1	2	3	4	5	6	7	8	9
国内乘客		T	T	T	T	F	F	F	F
头等舱		T	F	T	F	T	F	T	F
残疾乘客		F	F	T	T	F	F	T	T
行李重量 $W \leq 30\text{kg}$	T	F	F	F	F	F	F	F	F
免费	×								
$(W-30) \times 2$				×					
$(W-30) \times 3$					×				
$(W-30) \times 4$		×						×	
$(W-30) \times 6$			×						×
$(W-30) \times 8$						×			
$(W-30) \times 12$							×		



五、过程设计语言(PDL/伪码)

PDL的缺点是不如图形工具形象直观,描述复杂的条件组合与动作间的对应关系时,不如判定表清晰简单。

面向数据结构的设计方法

(1) Jackson方法和 Warnier方法是最著名的两个面向数据结构的设计方法

(2) Jackson图的优点：

a. 便于表示层次结构,而且是对结构进行自顶向下分解的有力工具

b. 形象直观可读性好

c. 既能表示数据结构也能表示程序结构(因为结构程序设计也只使用上述3种基本控制结构)

一、McCabe方法

(1) 结点：圆（一个圆可以代表一条或多条语句，程序流程图中的一个顺序的处理框序列和一个菱形判定框可以映射成流图中的一个结点）

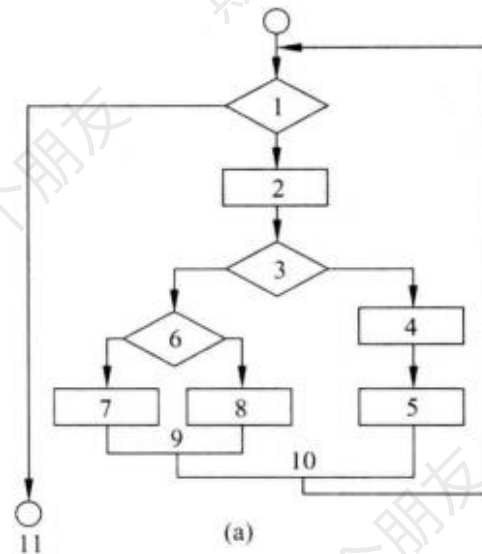
(2) 边：带箭头的线（可直可曲）

计算环形复杂度的**三种方法**：

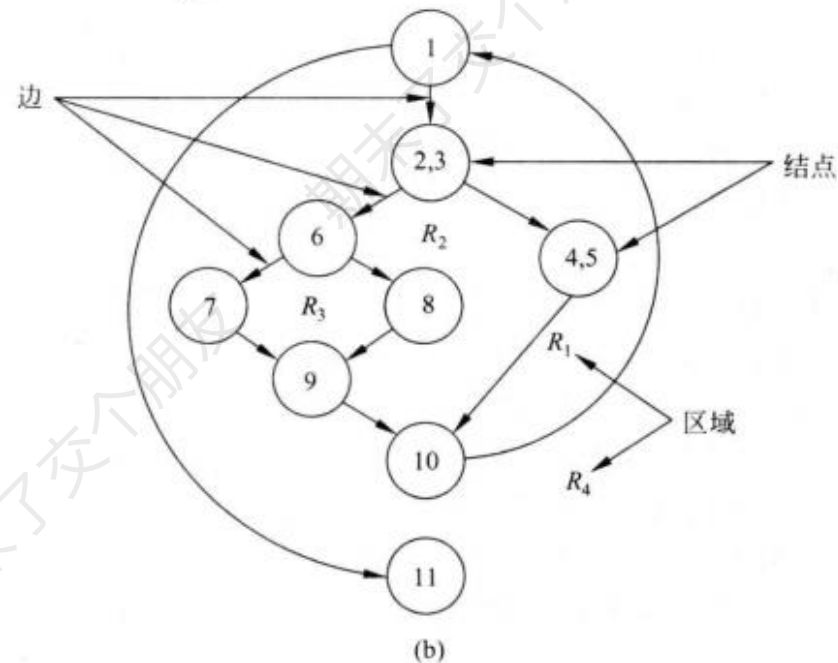
(1) 流图中线性无关的区域数等于环形复杂度

(2) 流图G的环形复杂度 $V(G) = E - N + 2$, 其中, E是流图中边的条数, N是结点数

(3) 流图G的环形复杂度 $V(G) = P + 1$, 其中, P是流图中判定结点的数目



(a)



(b)

二、Halstead 方法

(1) 令 N_1 为程序中运算符出现的总次数， N_2 为操作数出现的总次数，程序实际长度 N 定义为： $N=N_1+N_2$

(2) 令 n_1 为程序中使用的不同运算符(包括关键字)的个数， n_2 为不同操作数(变量和常数)的个数，则程序预测长度为 $H=n_1\log_2 n_1+n_2\log_2 n_2$

(3) 预测程序中包含错误的个数为 $E=N\log_2(n_1+n_2)/3000$ 。

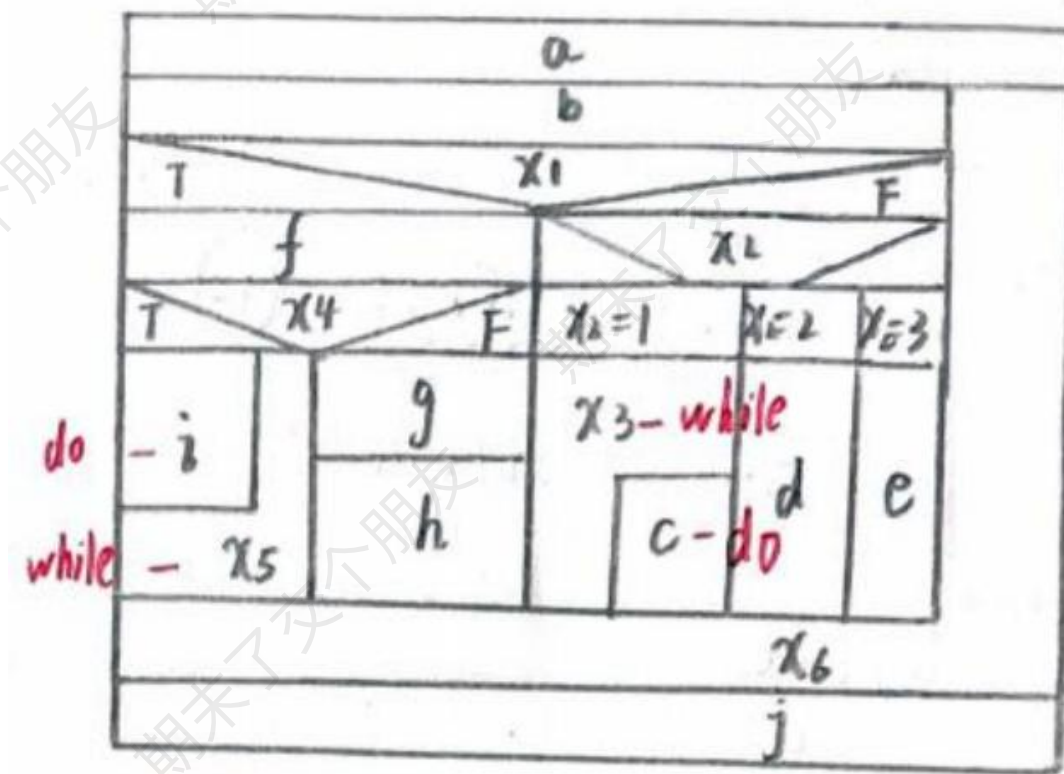
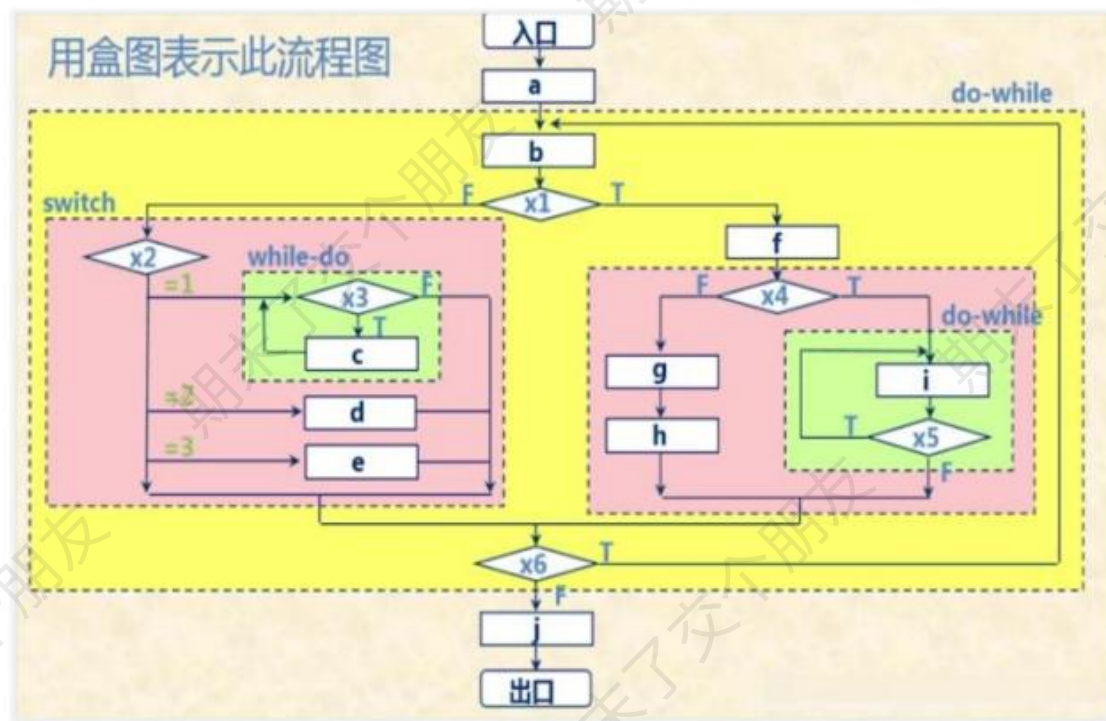
例题1 详细设计的基本任务是确定每个模块的（ ）设计

- A.功能
- B.调用关系
- C.输入输出数据
- D.算法

例题2 Jackson 设计方法是由英国的 M. Jackson 提出的，它是一种面向（ ）的软件设计方法。

- A.对象
- B.数据流
- C.数据结构
- D.控制结构

例题3



It's the end of the term, make a friend!



PASS EXAM

期末
必过

第七章 实现



(1)通常把编码和测试统称为实现

(2)软件测试在软件生命周期中横跨两个阶段。通常在编写出每个模块之后就对它做必要的测试(称为单元测试),模块的编写者和测试者是同一个人,编码和单元测试属于软件生命周期的同一个阶段。在这个阶段结束之后,对软件系统还应该进行各种综合测试,这是软件生命周期中的另一个独立的阶段,通常由专门的测试人员承担这项工作

(3)**选择程序设计语言时的实用标准：**

- a.系统用户的要求
- b.可以使用的编译程序
- c.可以得到的软件工具
- d.工程规模
- e.程序员的知识
- f.软件可移植性要求
- g.软件的应用领域

(4) 编码风格应该遵循的规则：

(1) 程序内部应该有很好的文档(程序内部的文档包括恰当的标识符、适当的注释和程序视觉组织)

(2) 数据说明应该易于理解和查阅

(3) 语句构造应该尽可能简单直观

(4) 输入输出风格遵守人机界面设计准则

(5) 效率满足用户需求即可

一、软件测试目标/定义(Myers给出)

- (1)测试是为了发现错误而执行程序的过程
- (2)好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案
- (3)成功的测试是发现了迄今为止尚未发现的错误的测试

二、软件测试准则

- (1)所有测试都应该能追溯到用户需求
- (2)应该远在测试开始之前就制定出测试计划
- (3)把 Pareto 原理应用到软件测试中(测试发现的错误中的 80%很可能是由程序中 20%的模块造成的)
- (4)应该从“小规模”测试开始，并逐步进行“大规模”测试
- (5)穷举测试是不可能的
- (6)为了达到最佳的测试效果，应该有独立的第三方从事测试工作

三、测试方法

(1)白盒测试：按照程序内部的逻辑测试程序，检测程序中的主要执行通路是否能按预定要求正确工作。**白盒测试又称为结构测试**

(2)黑盒测试：把程序看作一个黑盒子，完全不考虑程序的内部结构和处理过程。也就是说，黑盒测试是在程序接口进行的测试，它只检查程序功能是否按照规格说明书的规定正常使用，程序是否能适当地接收输入数据并产生正确的输出信息，程序运行过程中能否保持外部信息的完整性，**黑盒测试又称为功能测试**

四、测试步骤

(1)模块/单元测试

模块测试的目的是保证每个模块作为一个单元正确运行，又称为单元测试；在这个测试步骤中发现的往往是编码和详细设计的错误

(2)子系统测试

子系统测试把经过单元测试的模块放在一起形成一个子系统来测试；这个步骤着重测试模块的接口

(3)系统测试

系统测试是把经过测试的子系统装配成一个完整的系统来测试；在这个测试步骤中发现的往往是软件设计中的错误，也可能发现需求说明中的错误

四、测试步骤

(4)验收测试

验收测试把软件系统作为单一的实体进行测试，测试内容与系统测试的基本类似，但是它是在用户积极参与下进行的，而且可能主要使用实际数据进行测试。验收测试目的是验证系统确实能够满足用户的需要；在这个测试步骤中发现往往是系统需求说明书中的错误

(5)平行运行

所谓平行运行就是同时运行新开发出来的系统和将被它取代的旧系统，以便比较新旧两个系统的处理结果

(1)单元测试的**测试重点**：

- a.模块接口
- b.局部数据结构
- c.重要的执行通路
- d.出错处理通路
- e.边界条件

(2)**驱动程序**是一个“主程序”，它接收测试数据,把这些数据传送给被测试的模块,并且印出有关的结果

(3)**存根程序**代替被测试的模块所调用的模块。因此存根程序也可以称为“虚拟子程序”。它使用被它代替的模块的接口,可能做最少量的数据操作,印出对人口的检验或操作结果,并且把控制归还给调用它的模块

(1)定义：集成测试是测试和组装软件的系统化技术，**包括子系统测试和系统测试**。子系统测试把经过单元测试的模块放在一起形成一个子系统来测试；系统测试是把经过测试的子系统装配成一个完整的系统来测试

(2)模块组装成程序的两种方法：

a.先分别测试每个模块,再把所有模块按设计要求放在一起结合成所要的程序,这种方法称为**非渐增式测试**

b.把下一个要测试的模块同已经测试好的那些模块结合起来进行测试,测试完以后再把下一个应该测试的模块结合进来测试。这种每次增加一个模块的方法称为**渐增式测试(推荐)**

(3)集成测试中自顶向下和自底向上两种集成策略的优缺点：

1)自顶向下集成策略

优点：

具有错误隔离手段

能够在早期发现主要控制或主要抉择错误

不需要驱动程序

缺点：

潜在可重用代码得不到充分测试

需要存根程序

2)自底向上集成策略

优点：

具有错误隔离手段

潜在可重用代码能充分测试

不需要存根程序

缺点：

主要设计错误发现迟

需要驱动程序

(4)**回归测试**用于保证由于调试或其他原因引起的变化，不会导致非预期的软件行为或额外错误的测试活动

(1)**Alpha 测试**：由用户在开发者的场所进行，并且在开发者对用户的“指导”下进行测试。开发者负责记录发现的错误和使用中遇到的问题。总之，Alpha 测试是在受控的环境中进行的

(2)**Beta 测试**：由软件的最终的用户们在一个或多个客户场所进行。与 Alpha 测试不同，开发者通常不在 Beta 测试的现场，因此，Beta 测试是软件在开发者不能控制的环境中的真实应用

一、逻辑覆盖

(1)语句覆盖：使每个语句至少执行一次

(2)判定（分支）覆盖：使每个判定的**每个分支**都至少执行一次

(3)条件覆盖：使每个判定的**每个条件**都取到各种可能的结果

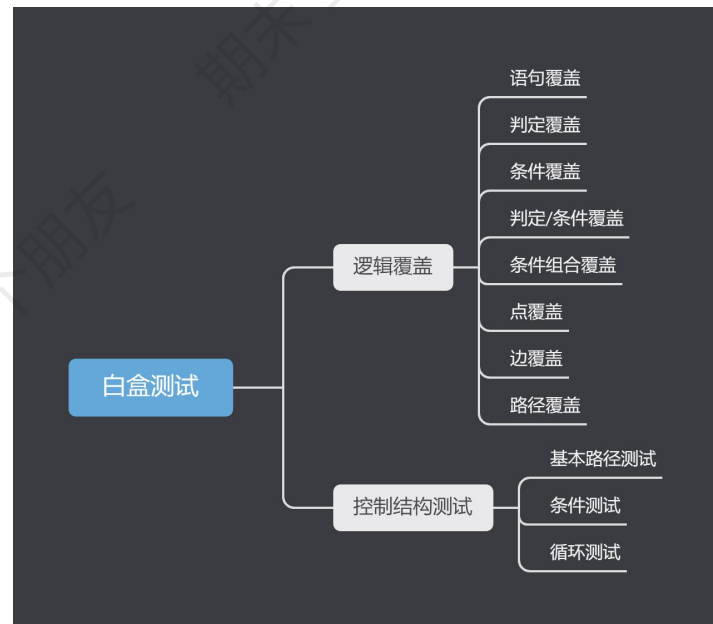
(4)判定/条件覆盖：使判定表达式中的每个条件都取到各种可能的值，而且每个判定表达式也都取到各种可能的结果

(5)条件组合覆盖：使每个判定表达式中条件的各种可能组合都至少出现一次

(6)点覆盖：和语句覆盖相同

(7)边覆盖：和判定(分支)覆盖相同

(8)路径覆盖：使程序的每条可能路径都至少执行一次



二、基本路径测试

- a. 根据过程设计结果画出相应数据流图
- b. 计算流图的环形复杂度
- c. 确定线性独立路劲的基本集合
- d. 设计可强制执行基本集合中每条路径的测试用例

黑盒测试

```
graph LR; A[黑盒测试] --- B[等价划分]; A --- C[边界值分析]; A --- D[错误推测];
```

等价划分

边界值分析

错误推测

(1)**定义**：调试就是把症状和原因联系起来的尚未被深入认识的智力过程(调试是在测试发现错误之后排除错误的过程)

(2)**调试工作困难的原因**：(调试工作是软件开发过程中最艰巨的力劳动)

- a. 症状和产生症状的原因可能在程序中相距甚远
- b. 当改正了另一个错误之后，症状可能暂时消失了
- c. 症状可能实际上并不是由错误引起的
- d. 症状可能由不易追踪的人为错误引起的
- e. 症状可能由定时问题而不是由处理问题引起
- f. 可能很难重新产生完全一样的输入条件
- g. 症状可能时有时无
- h. 症状可能是由分布在许多任务中的原因引起的，这些任务运行在不同的处理机上

(3)**调试的途径**：

- a. 蛮干法 b. 回溯法 c. 原因排除法(对分查找法 / 归纳法 / 演绎法)

- (1)软件可靠性：程序在给定的时间间隔内，按照规格说明书的规定成功运行的概率。
- (2)软件可用性：程序在给定的时间点，按照规格说明书的规定，成功地运行的概率。
- (3)**区别**：可靠性和可用性之间的主要差别是，可靠性意味着在 0 到 t 这段时间间隔内系统没有失效，而可用性只意味着在时刻 t ,系统是正常运行的。

(4)平均无故障时间MTTF

$$MTTF = \frac{1}{K(E_T/I_T - E_c(\tau)/I_T)}$$

E_T ——测试之前程序中错误总数；

I_T ——程序长度(机器指令总数)；

τ ——测试(包括调试)时间；

$E_d(\tau)$ ——在 0 至 τ 期间发现的错误数；

$E_c(\tau)$ ——在 0 至 τ 期间改正的错误数。

例题1 覆盖准则最强的是 ()

A.语句覆盖

B.判定覆盖

C.条件覆盖

D.路径覆盖

例题2 () 方法需要考察模块间的接口和各个模块直接的关系。

A.单元测试

B.集成测试

C.确认测试

D.系统测试

例题3

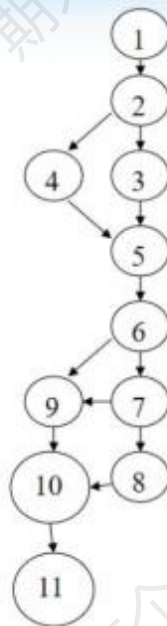
已知下列伪码，请用基本路径测试方法设计测试用例。

- 要求：（1）写出完整设计过程
（2）控制流图中结点数目和题目中标定的一致（即无需合并语句）

```

STRAT
1:      INPUT (A, B, C)
2:      IF (A<0)
3:          THEN X=A-B
4:          ELSE X=A+B
5:      ENDIF
6:      IF (B>A)
7:          AND (C>1)
8:          THEN Y=C+A
9:          ELSE Y=C-B
10:     ENDIF
11:     PRINT (X, Y)
STOP
  
```

(1) 控制流图如下：



(2) 环形复杂度计算

$V(G) = \text{区域数} = 4$

$V(G) = \text{边数} - \text{结点数} + 2 = 4$

$V(G) = \text{判定结点的数目} + 1 = 4$

(3) 独立路径

1) 1-2-4-5-6-9-10-11

2) 1-2-3-5-6-9-10-11

3) 1-2-3-5-6-7-9-10-11

4) 1-2-3-5-6-7-8-10-11

(4) 测试用例

1) 输入: $A \geq 0$, $B \leq A$, C 任意

输出: $X=A$, $Y=C-B$

2) 输入: $A < 0$, $B \leq A$, C 任意

输出: $X=-A$, $Y=C-B$

3) 输入: $A < 0$, $B > A$, $C \leq 1$

输出: $X=-A$, $Y=C-B$

4) 输入: $A \geq 0$, $B > A$, $C > 1$

输出: $X=-A$, $Y=C+B$

注意: 根据上面范围, 给出具体数值也可以

例题4

对一个包含 10 000 条机器指令的程序进行一个月集成测试后,总共改正了 15 个错误,此时 $MTTF=10h$;经过两个月测试后,总共改正了 25 个错误(第二个月改正了 10 个错误), $MTTF=15h$ 。

要求:

(1) 根据上述数据确定 $MTTF$ 与测试时间之间的函数关系,画出 $MTTF$ 与测试时间 τ 的关系曲线。在画这条曲线时你做了什么假设?

(2) 为做到 $MTTF=100h$,必须进行多长时间的集成测试? 当集成测试结束时总共改正了多少个错误? 还有多少个错误潜伏在程序中?

答: (1) 假设在程序的平均无故障时间 $MTTF$ 和测试时间 τ 之间存在线性关系,即

$$MTTF=a+b\tau$$

根据题意可知,当 $\tau=1$ 时 $MTTF=10$,当 $\tau=2$ 时 $MTTF=15$,把这些已知的数据代入上列方程后得到下列的联立方程式

$$\begin{cases} a+b=10 \\ a+2b=15 \end{cases}$$

解上列联立方程得出 $a=5, b=5$ 。

因此, $MTTF$ 与 τ 之间有下列关系

$$MTTF=5+5\tau$$

根据上列方程式画出平均无故障时间 $MTTF$ 与测试时间 τ 的关系曲线,如图 4.2 所示。

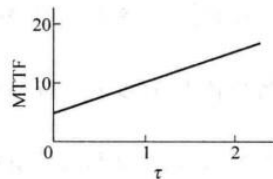


图 4.2 平均无故障时间 $MTTF$ 与测试时间 τ 的关系

例题4

对一个包含 10 000 条机器指令的程序进行一个月集成测试后,总共改正了 15 个错误,此时 $MTTF=10h$;经过两个月测试后,总共改正了 25 个错误(第二个月改正了 10 个错误), $MTTF=15h$ 。

要求:

(1) 根据上述数据确定 $MTTF$ 与测试时间之间的函数关系,画出 $MTTF$ 与测试时间 τ 的关系曲线。在画这条曲线时你做了什么假设?

(2) 为做到 $MTTF=100h$,必须进行多长时间的集成测试? 当集成测试结束时总共改正了多少个错误? 还有多少个错误潜伏在程序中?

(2) 为使 $MTTF=100h$,需要的测试时间可由下面的方程式得出

$$100=5+5\tau$$

解上列方程式得

$$\tau=19$$

即,需要进行 19 个月的集成测试。

已知平均无故障时间与单位长度程序中剩余的错误数成反比,具体到本题程序即有

$$MTTF = \frac{10\,000}{K(E_T - E_c(\tau))}$$

根据题意可知,改正了 15 个错误后 $MTTF=10$,改正了 25 个错误后 $MTTF=15$,把这些已知的数据代入上列方程之后,得到下列的联立方程式

$$\begin{cases} 10 = \frac{10\,000}{K(E_T - 15)} \\ 15 = \frac{10\,000}{K(E_T - 25)} \end{cases}$$

解上列联立方程式得到 $E_T=45, K=33.33$ 。

已知当集成测试结束时 $MTTF=100h$,即

$$100 = \frac{10\,000}{33.33(45 - E_c(19))}$$

所以

$$E_c(19) = 42$$

也就是说,当集成测试结束时总共改正了 42 个错误,还有 $45 - 42 = 3$ 个错误潜伏在程序中。

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第八章 维
护



软件维护的定义

(1)定义：软件维护就是在软件已经交付使用之后，为了改正错误或满足新的需要而修改软件的过程。

(2)分类：

- a. **改正性维护**：诊断和改正错误的过程称为改正性维护
- b. **适应性维护**：为了和变化了的环境适当地配合而进行的修改软件的活动,是既必要又经常的维护活动
- c. **完善性维护**：通常，在使用软件的过程中由用户提出增加新功能或修改已有功能的建议，还可能提出一般性的改正意见。为了满足这类需求的维护叫做完善性维护(**使用最多的是完善性维护**)
- d. **预防性维护**：为了改进未来的可维护性或可靠性，或为了给未来的改进奠定更好的基础而修改软件的维护活动称为预防性维护

(1)结构化维护和非结构化维护差别巨大

- a. 非结构化维护的代价很高,这种维护方式是没有使用软件工程方法学开发出来的软件的必然结果
- b. 以完整的软件配置为基础的结构化维护,是在软件开发过程中应用软件工程方法学的结果。虽然有了软件的完整配置,但并不能保证维护时没有问题,但是确实能减少精力的浪费并,提高维护的总体质量

(2)维护的代价高昂

- a. 因为可用的资源必须供维护任务使用,以至耽误甚至丧失了开发的良机
- b. 当看来合理的有关改错或修改的要求不能及时满足时将引起用户的不满
- c. 由于维护时的改动,在软件中引入了潜伏的错误,从而降低了软件的质量
- d. 当必须把软件工程师调去从事维护工作时,将在开发过程中造成混乱
- e. 软件维护的最后一个代价就是生产率大幅度下降

(3)维护的问题很多

- a. 理解别人写的程序通常非常困难
- b. 需要维护的软件往往没有合格的文档
- c. 当要求对软件进行维护时，不能指望由开发人员给人们仔细说明软件
- d. 绝大多数软件在设计时没有考虑将来的修改
- e. 软件维护不是一项吸引人的工作

软件维护过程

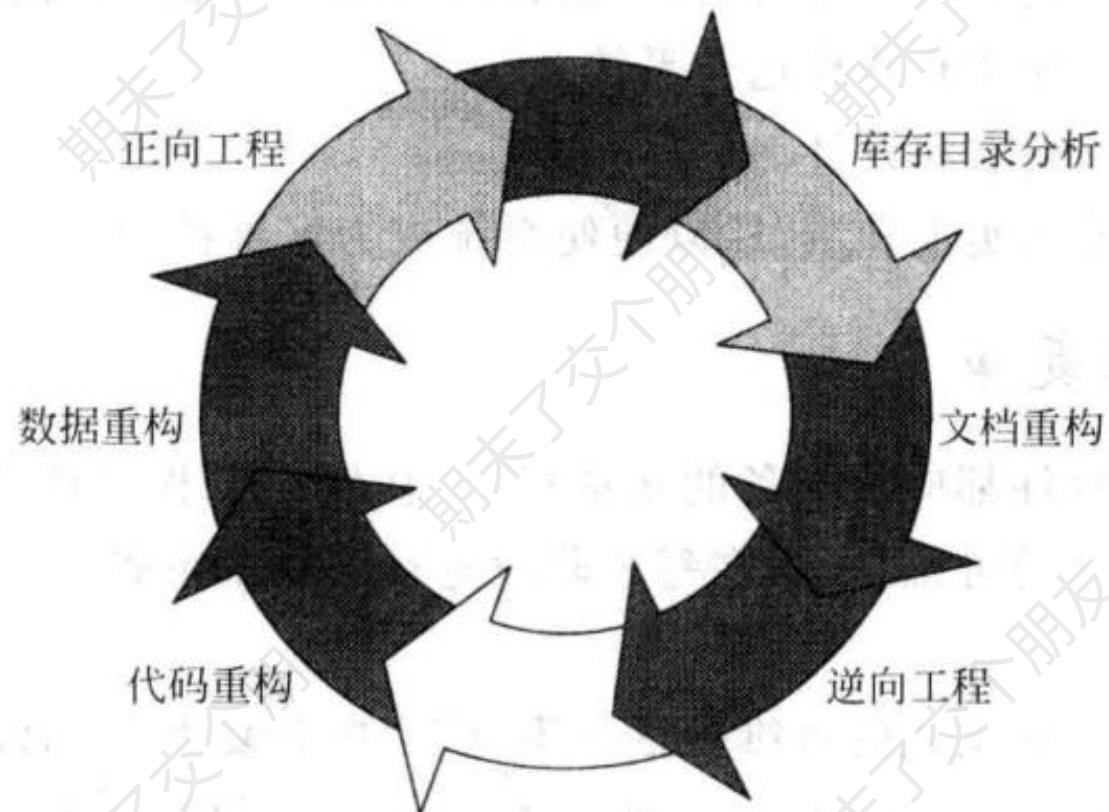
- 维护组织
- 维护报告
- 维护的事件流
- 保存维护记录
- 评价维护活动

(1)定义：可维护性是指维护人员理解、改正、改动或改进这个软件的难易程度

(2)决定软件可维护性的因素：

- **可理解性**:读者理解软件的难易程度
- **可测试性**:论证程序正确性的容易程度
- **可修改性**:程序容易修改的程度
- **可移植性**:把程序从一种计算环境（硬件配置和操作系统）转移到另一种计算环境的难易程度
- **可重用性**:同一个软件不做修改或稍加改动，就可以在不同环境中多次重复使用

(3)**文档是影响软件可维护性的决定因素**。软件系统的文档可以分为**用户文档和系统文档**两类。用户文档主要描述系统功能和使用方法,并不关心这些功能是怎样实现的;系统文档描述系统设计、实现和测试等各方面的内容。



例题1 软件生命周期中所花费费用最多的阶段是 ()

A.详细设计

B.软件编码

C.软件测试

D.软件维护

例题2 软件维护工作的最主要部分是 ()。

A .纠正性维护

B.适应性维护

C.完善性维护

D.预防性维护

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第九章
论

面向对象方法学引



一、面向对象方法学要点

- 认为客观世界是由各种对象组成的，任何事物都是对象，复杂的对象可以由比较简单的对象以某种方式组合而成，面向对象方法用对象分解取代了传统方法的功能分解
- 把所有对象都划分成各种对象类，每个对象类都定义了一组数据和一组方法
- 按照子类与父类的关系，把若干个对象类组成一个层次结构的系统
- 对象彼此之间仅能通过传递消息互相联系

面向对象就是既使用对象又使用类和继承等机制,而且对象之间仅能通过传递消息实现彼此通信

面向对象 = 对象 + 类 + 继承 + 通信

二、面向对象方法学优点

- 与人类习惯的思维方式一致
- 稳定性好
- 可重用性好
- 较易开发大型软件产品
- 可维护性好

面向对象的概念

(1)对象的定义：对象是封装了数据结构及可以施加在这些数据结构上的操作的封装体，这个封装体有可以唯一地标识它的名字，而且向外界提供一组服务

(2)对象的特点：

- a. 以数据为中心
- b. 对象是主动的
- c. 实现了数据封装
- d. 本质上具有并行性
- e. 模块独立性好

(3)类的定义：具有相同属性和行为的一组对象的集合

(4)实例的定义：由某个特定的类所描述的一个具体对象

(5)消息的定义：要求某个对象执行在定义它的那个类中所定义的某个操作的规格说明。

(6)方法的定义：对象所能执行的操作，也就是类中所定义服务

(7)属性的定义：类中定义的数据，是对客观世界实体的抽象表达

(8)**封装**的定义：将对象的属性和方法捆绑成一个逻辑单元，并尽可能的隐藏内部的实现细节，使得外部对状态的访问或修改仅能通过封装提供的接口进行

(9)**继承**的定义：广义地说，继承是指能够直接获得已有的性质和特征，而不必重复定义它们。在面向对象的软件技术中，继承是子类自动地共享基类中定义的数据和方法的机制

(10) **多态**的定义：指子类对象可以像父类对象那样使用,同样的消息既可以发送给父类对象也可以发送给子类对象。也就是说,在类等级的不同层次中可以共享(公用)一个行为(方法)的名字,然而不同层次中的每个类却各自按自己的需要来实现这个行为。当对象接收到发送给它的消息时,根据该对象所属于的类动态选用在该类中定义的实现算法

(11)重载的定义：

a. 函数重载：在同一作用域内的若干参数特征不同的函数，可以使用相同函数名

b. 运算符重载：同一运算符可以施加在不同类型的操作数上面

- 描述系统数据结构的**对象模型** -- **类图**表示
- 描述系统控制结构的**动态模型** -- **状态图**表示
- 描述系统功能的**功能模型** -- **用例图/数据流图**表示

在任何情况下,对象模型始终都是最重要、最基本、最核心的

(1)对象模型的作用：表示静态的、结构化的系统的数据性质。它是对模拟客观世界实体的对象以及对象彼此间的关系的映射，描述了系统的静态结构

(2)类与类之间的关系：**关联、泛化(继承)、依赖和细化4种关系**

a.关联表示两个类的对象之间存在某种语义上的联系

b.聚集是关联的特例，分为**共享聚集和组合聚集**。如果在聚集关系中处于部分方的对象可同时参与多个处于整体方对象的构成，则该聚集称为共享聚集；如果部分类完全隶属于整体类，部分与整体共存，整体不存在了部分也会随之消失，则该聚集称为组合聚集

(2)类与类之间的关系：**关联、泛化(继承)、依赖和细化**4种关系

c.泛化分为**普通泛化和受限泛化**，其中普通泛化即继承，受限泛化可以给泛化关系附加约束条件，以进一步说明该泛化关系的使用方法或扩充方法

d.依赖关系描述两个模型元素之间的语义连接关系：其中一个模型元素是独立的，另一个模型元素不是独立的，它依赖于独立的模型元素，如果独立的模型元素改变了，将影响依赖于它的模型元素

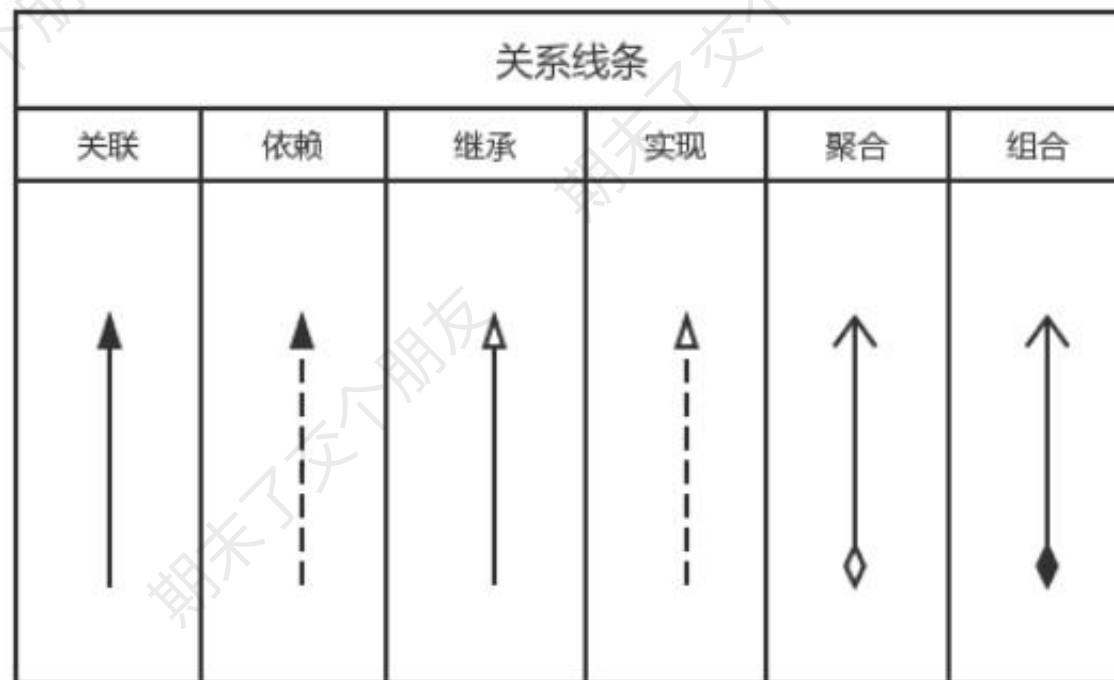
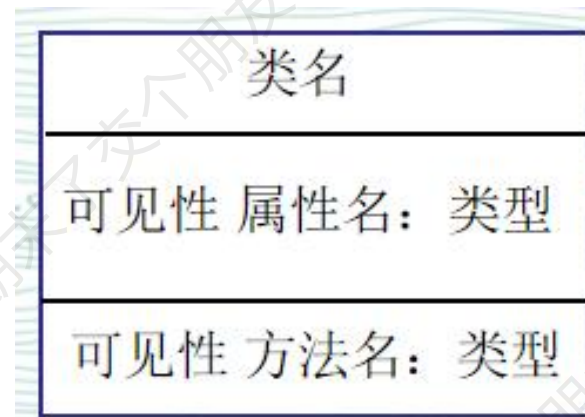
e.当对同一事物在不同抽象层次上描述时，这些描述具有细化关系。

聚集：整体-部分关系

泛化：一般-特殊关系

(1)类的图形符号为长方形，用两条横线把长方形分成上、中、下3个区域(下面两个区域可省略)，3个区域分别放类的名字、属性和服务

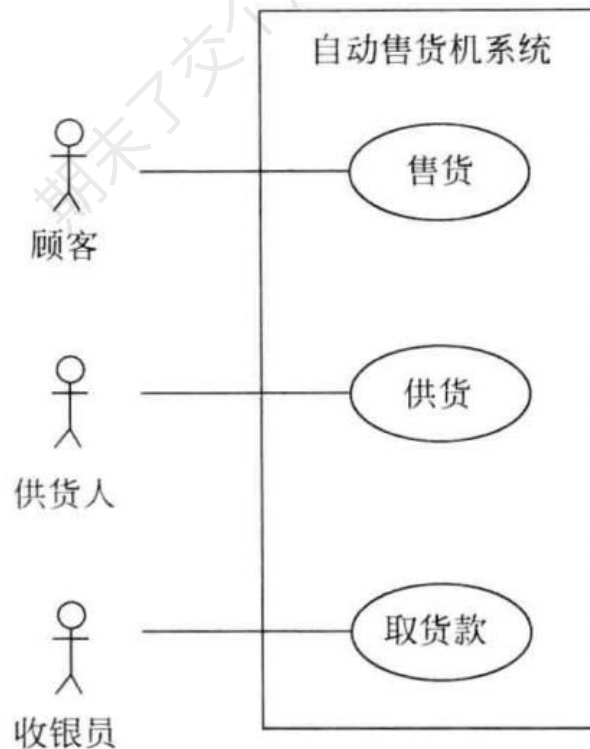
(2)属性和方法的可见性(即可访问性)通常有下述3种:公有的(public)、私有的(private)和保护(protected),分别用加号(+)、减号(-)和井号(#)表示



动态模型的作用：表示瞬时的、行为化的系统的控制性质，它规定了对象模型中对象的合法变化序列

(1)功能模型的作用：表示软件系统的功能性质，它指明了系统应该做什么，更直接地反映了用户对目标系统的需求

(2)用例图包含的模型元素有系统、行为者,用例及用例之间的关系。方框代表系统,椭圆代表用例,线条人代表行为者,它们之间的连线表示关系



3种模型之间的关系

- (1) 针对每个类建立的动态模型,描述了类实例的生命周期或运行周期
- (2) 状态转换驱使行为发生,这些行为在数据流图中被映射成处理,在用例图中被映射成用例,它们同时与类图中的服务相对应
- (3) 功能模型中的处理(或用例)对应于对象模型中的类所提供的服务
- (4) 数据流图中的数据存储,以及数据的源点/终点,通常是对象模型中的对象
- (5) 数据流图中的数据流,往往是对象模型中对象的属性值,也可能是整个对象
- (6) 用例图中的行为者,可能是对象模型中的对象
- (7) 功能模型中的处理(或用例)可能产生动态模型中的事件
- (8) 对象模型描述了数据流图中的数据流、数据存储以及数据源点/终点的结构

例题1 在 UML 的各类图中 () 是描述软件功能的。

A.用例图

B.类图

C.活动图

D.顺序图

例题2 面向对象分析过程中建立的模型有 ()

A.数据模型、功能模型、活动模型

B.对象模型、功能模型、测试模型

C.属性模型、功能模型、对象模型

D.对象模型、功能模型、动态模型

例题3

某校图书馆管理系统具有以下功能。

- (1)借书:先为读者办理借书证,借书证上记录读者姓名、学号、所属系和班级等信息。借书时根据读者的借书证查阅读者档案,若借书数 H 未超过规定数量,则办理借阅手续,修改库存记录及读者档案;若超过规定数量则不予借阅。
- (2)还书:根据读者书中的条形码,修改库存记录及读者档案,若借阅时间超过规定期限则罚款。
- (3)图书管理员还要定期生成订书清单,包括书名、图书代号、单价、数量等,根据需向供应商订购图书。

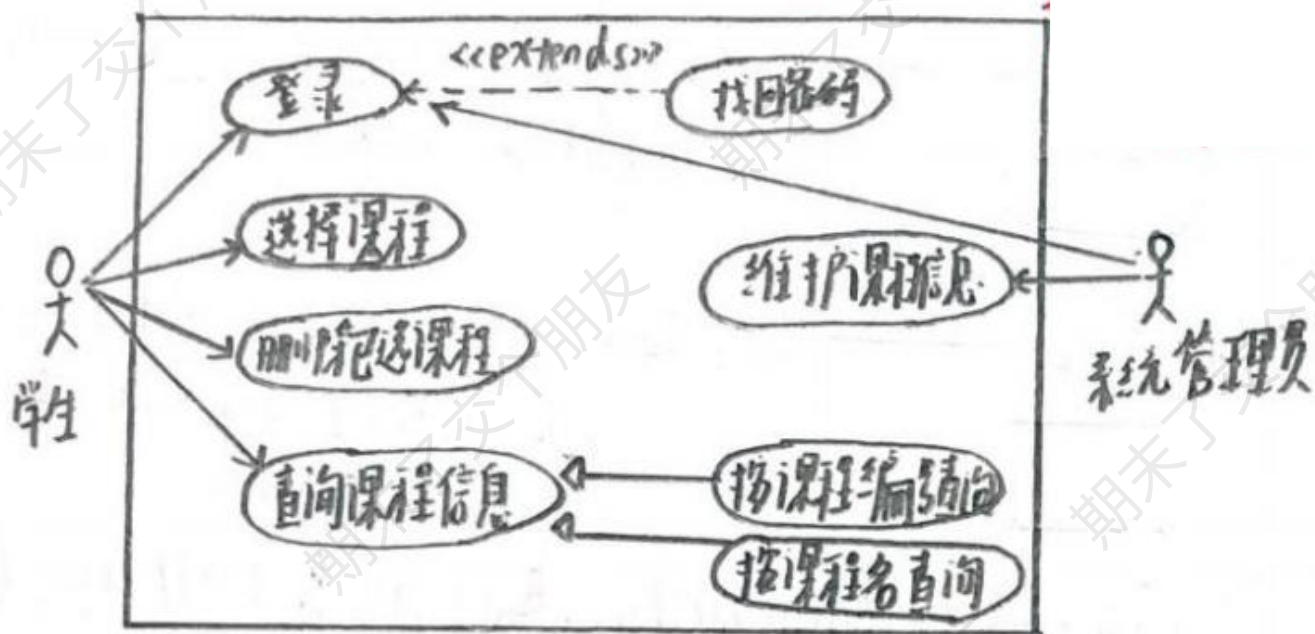
请按照以上需求建立这个图书馆管理的对象模型。



例题4 试画出网上选课系统的用例图。

用例有：登录；找回密码；查询课程信息；按课程编号查询；按课程名查询；选择课程；删除已选课程；维护已选课信息。

参与者有：系统管理员和学生。



It's the end of the term, make a friend!



PASS EXAM

期末
必过

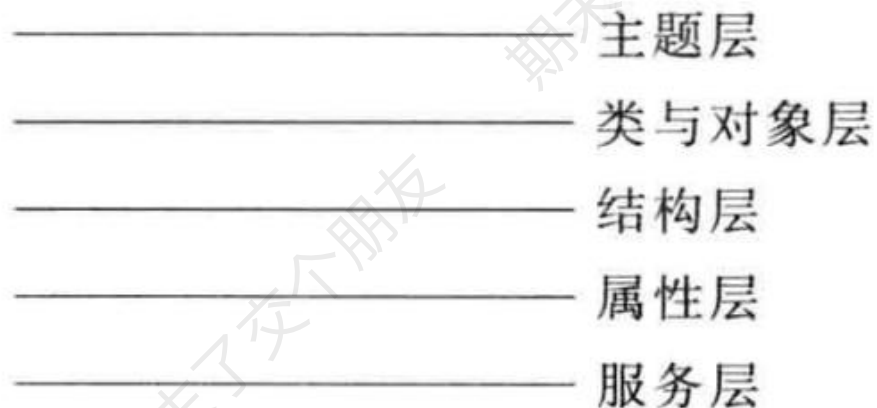
第十章 面向对象分析



面向对象分析的基本过程

(1)面向对象分析大体上按照下列顺序进行：寻找类与对象，识别结构，识别主题，定义属性，建立动态模型，建立功能模型和定义服务

(2)面向对象设计模型五个层次：主题、类与对象、结构、属性和服务



需求陈述书写要点

- (1)需求陈述的内容应该包括问题范围、功能需求、性能需求、应用环境及建设条件
- (2)需求陈述应该阐明“做什么”而不是“怎样做”
- (3)它应该描述用户的需求而不是提出解决问题的办法
- (4)应该指出哪些是系统必要的性质，哪些是任选的性质
- (5)应该避免对设计策略施加过多的约束，也不要描述系统的内部结构，因为这样做将会限制实现的灵活性

It's the end of the term, make a friend!



PASS EXAM

期末
必过

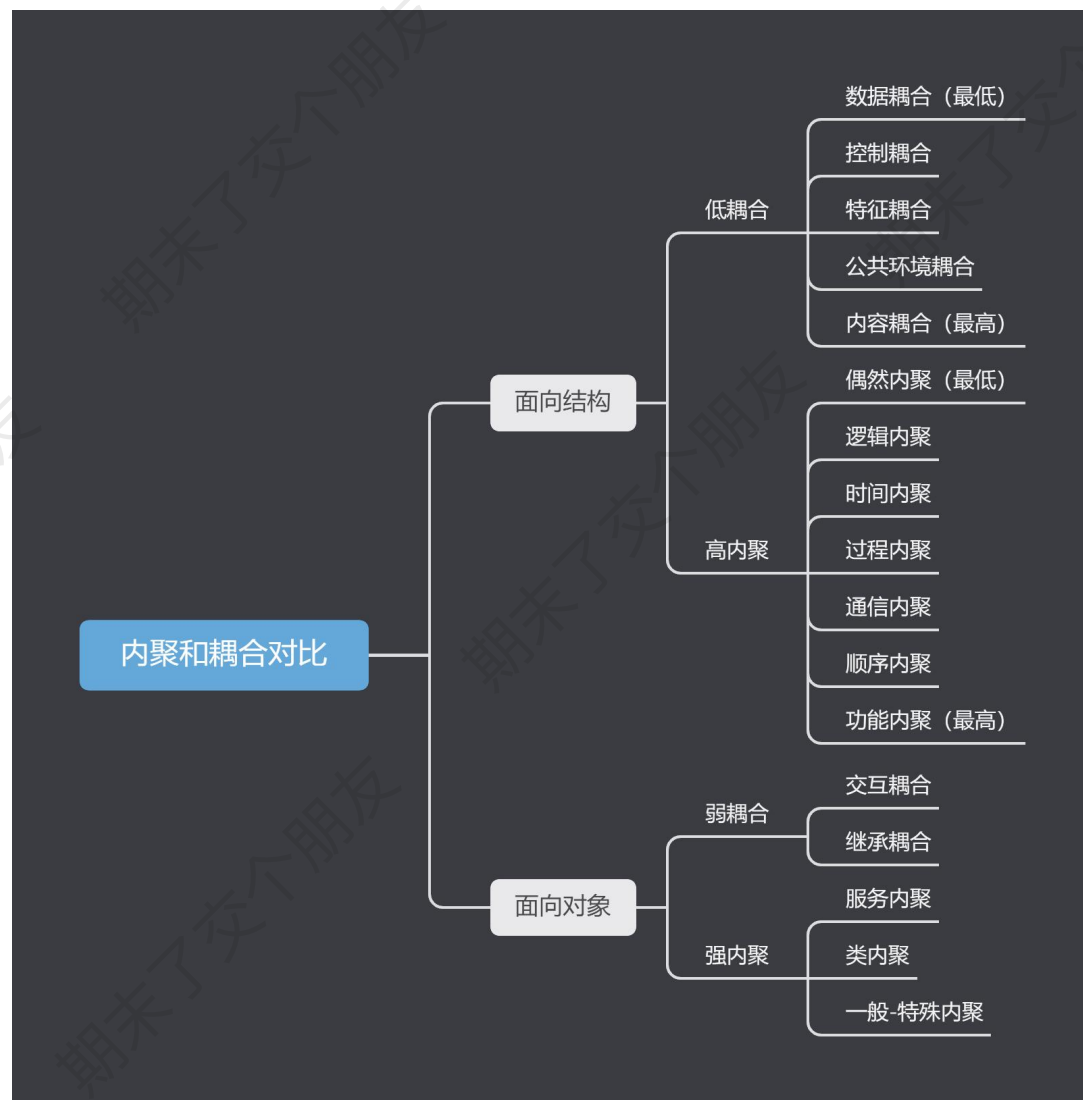
第十一章 面向对象设计



01

面向对象设计的准则

- 模块化
- 抽象
- 信息隐藏
- 弱耦合(交互耦合和继承耦合)
- 强内聚(服务内聚、类内聚和一般 - 特殊内聚)
- 可重用



启发规则

- 设计结果应该清晰易懂
- 一般 - 特殊结构的深度应适当
- 设计简单的类
- 设计简单的类
- 使用简单的服务
- 把设计变动减至最小

启发规则

面向结构

改进软件结构提高模块独立性

模块规模应该适中

深度、宽度、扇出和扇入都应适当

模块的作用域应该在控制域之内

力争降低模块接口的复杂程度

设计单入口单出口的模块

模块功能应该可以预测

面向对象

设计结果应该清晰易懂

一般-特殊结构的深度应适当

设计简单的类

使用简单的协议

使用简单的服务

把设计变动减至最小

(1)**重用的定义**：重用是指同一事物不作修改或稍加改动就多次重复使用

(2)**重用的三个层次**

- a.知识重用
- b.方法和标准重用
- c.软件成分的重用

(3)**软件成分的重用的三个级别**

- a.代码重用(调用库中的模块)
- b.设计结果重用(重用某个软件系统的设计模型即求解域模型)
- c.分析结果重用(重用某个系统的分析模型)

(4)**典型的软件可重用成分**：项目计划、成本估计、体系结构、需求模型和规格说明、设计、源代码、用户文档和技术文档、用户界面、数据、测试用例

(5)**可重用软构件应具备的特点**

- a. 类构件模块独立性强
- b. 类构件具有高度可塑性
- c. 类构件接口清晰、简明、可靠

(6)类构件的三种重用方式：

a. 实例重用

由于类的封装性，使用者无须了解实现细节就可以使用适当地构造函数，按照需要创建类的实例。然后向所创建的实例发送适当消息，启动相应服务，完成需要完成的工作。这是最基本的重用方式。此外，还可以用几种简单的对象作为类的成员创建一个更复杂的类，这是实例重用的另一种形式

b. 继承重用

面向对象方法特有的继承性提供了一种对已有的类构件进行裁剪的机制。当已有的类构件不能通过实例重用完全满足当前系统需求时，继承重用提供了一种安全地修改已有类构件，以便在当前系统中重用的手段

c. 多态重用

多态重用利用多态性不仅可以使对象的对外接口更加一般化，从而降低了消息连接的复杂程度，而且还提供了一种简便可靠的软构件组合机制。系统运行时，根据接收消息的对象类型，由多态性机制启动正确的方法，去响应一个一般化的消息，从而简化了消息界面和软件构件的连接过程

(1)四个子系统

- a.问题域子系统
- b.人机交互子系统
- c.任务管理子系统
- d.数据管理子系统

(2)子系统之间的两种交互方式

- a.客户-供应商关系
- b.平等伙伴关系

(3)组织系统的两种方案

- a.层次组织（水平划分）：把软件系统组织成一个层次系统，每层是一个子系统，上层在下层的基础上建立，下层为上层功能而提供必要的服务，每一层内包含的对象彼此相互独立，而处于不同层次上的系统相互关联。
- b.块状组织（垂直划分）：把软件系统垂直地分解成若干个相对独立的、弱耦合的子系统，一个子系统相当于一块，每块提供一种类型的服务。

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第十二章 面向对象实
现



(1)面向对象语言的优点

- a. 一致的表示方法
- b. 可重用性
- c. 可维护性

(2)面向对象语言的技术特点

- a. 支持类与对象的概念
- b. 实现整体 - 部分结构的机制
- c. 实现一般 - 特殊结构的机制
- d. 实现属性和服务的机制
- e. 类型检查
- f. 丰富的类库
- g. 高效率
- h. 持久保存对象
- i. 参数化类
- j. 方便的开发环境

(3)选择面向对象语言考虑因素

- a. 将来能否占主导地位
- b. 可重用性
- c. 类库和开发环境
- d. 对用户学习面向对象分析、设计和编码技术所能提高的培训服务
- e. 在使用这个面向对象语言期间能提供的技术支持
- f. 能提供给开发人员使用的开发工具、开发平台、发行平台
- g. 对机器性能和内存的需求
- h. 集成已有软件的容易程度

良好的面向对象程序设计风格

(1)提高可重用性

- a. 提高方法的内聚
- b. 减小方法的规模
- c. 保持方法的一致
- d. 把策略和实现分开
- e. 全面覆盖
- f. 尽量不使用全局信息
- g. 利用继承机制

(2)提高可扩充性

- a. 封装实现策略
- b. 不要用一个方法遍历多条关联链
- c. 避免使用多分支语句
- d. 精心确定公有方法

(3)提高健壮性

- a. 预防用户的操作错误
- b. 检查参数的合法性
- c. 不要预先确定限制条件
- d. 先测试后优化

面向对象的集成测试策略

(1)**基于线程的测试**。这种策略把响应系统的一个输入或一个事件所需要的那些类集成起来。分别集成并测试每个线程，同时应用回归测试以保证没有产生副作用

(2)**基于使用的测试**。这种方法首先测试几乎不使用服务器类的那些类(称为独立类),把独立类都测试完之后,再测试使用独立类的下一个层次的类(称为依赖类)。对依赖类的测试一个层次一个层次地持续进行下去,直至把整个软件系统构造完为止

It's the end of the term, make a friend!



PASS EXAM

期末
必过

第十三章 软件项目管
理



一、代码行技术

(1)**基本思想**：代码行技术是比较简单的定量估算软件规模的方法。这种方法依据以往开发类似产品的经验和历史数据，估计实现一个功能所需要的源程序行数

(2)**优点**：代码是所有软件开发项目都有的“产品”，而且很容易计算代码行数

(3)**缺点**

a.源程序仅是软件配置的一个成分,用它的规模代表整个软件的规模似乎不太合理

b.用不同语言实现同一个软件所需要的代码行数并不相同

c.这种方法不适用于非过程语言

(4)**计算公式(单位LOC/KLOC)**

$$L = \frac{\bar{a} + 4\bar{m} + \bar{b}}{6}$$

二、功能点技术

(1)**基本思想**：功能点技术依据对软件信息域特性和软件复杂性的评估结果，估算软件规模

(2)**优点**：功能点数与所用的编程语言无关,看起来功能点技术比代码行技术更合理一些。

(3)**缺点**：在判断信息域特性复杂级别和技术因素的影响程度时,存在着相当大的主观因素

(4)**计算公式(单位FP)**

$$UFP = a_1 \times Inp + a_2 \times Out + a_3 \times Inq + a_4 \times Maf + a_5 \times Inf$$

$$DI = \sum_{i=1}^{14} F_i$$

$$TCF = 0.65 + 0.01 \times DI$$

二、功能点技术

表 13.1 信息域特性系数值

特性系数 \ 复杂级别	简单	平均	复杂
输入系数 a_1	3	4	6
输出系数 a_2	4	5	7
查询系数 a_3	3	4	6
文件系数 a_4	7	10	15
接口系数 a_5	5	7	10

表 13.2 技术因素

序号	F_i	技术因素
1	F_1	数据通信
2	F_2	分布式数据处理
3	F_3	性能标准
4	F_4	高负荷的硬件
5	F_5	高处理率
6	F_6	联机数据输入
7	F_7	终端用户效率
8	F_8	联机更新
9	F_9	复杂的计算
10	F_{10}	可重用性
11	F_{11}	安装方便
12	F_{12}	操作方便
13	F_{13}	可移植性
14	F_{14}	可维护性

(1) 工作量是软件规模(KLOC或FP)的函数,工作量的单位通常是人月(pm)

(2) 估算模型

- 静态单变量模型
- 动态多变量模型
- COCOMO2模型(COCOMO是构造性成本模型)

(1)开发小组规模的扩大个人生产率将下降的原因

- 当小组变得更大时，每个人需要更多时间与组内其他成员讨论问题、协调工作，因此增加了通信开销
- 如果在开发过程中增加小组成员，则最初一段时间内项目组总生产率不仅不会提高反而会下降。这是因为新成员在开始时不仅不是生产力，而且在他们学习期间还需要花费小组其他成员的时间

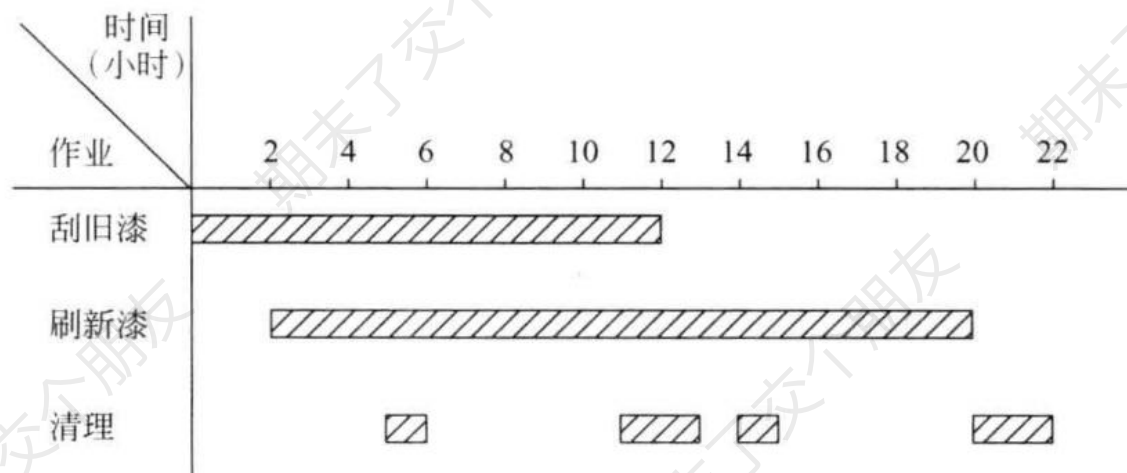
(2)制定进度计划的工具

- 甘特图
- 工程网络

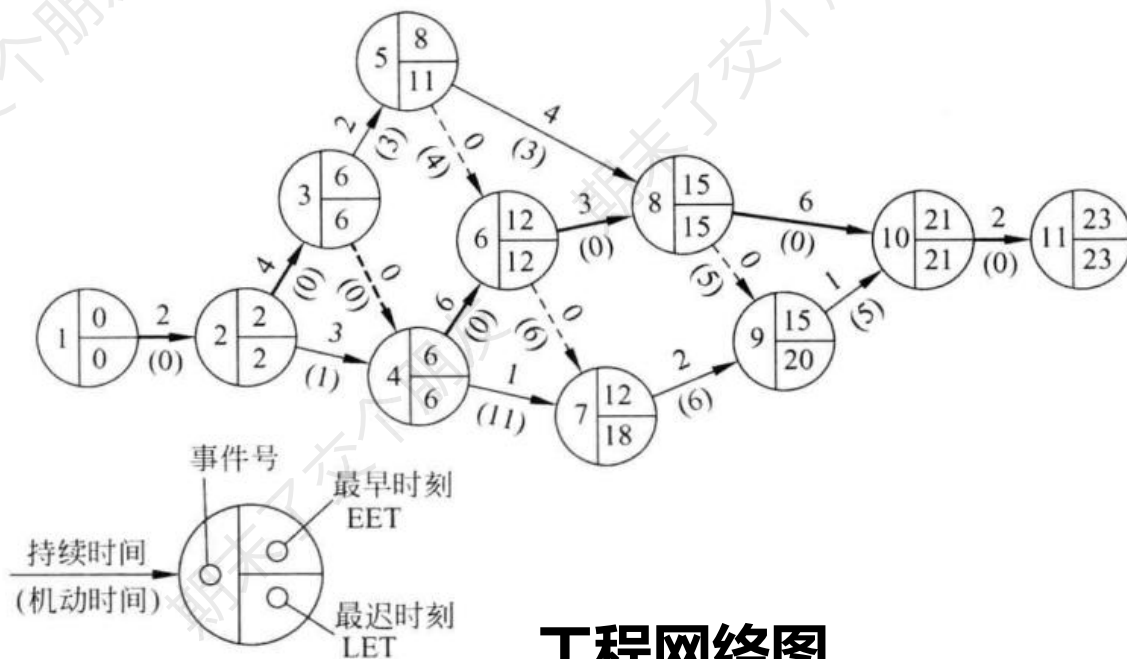
工序 墙壁	刮旧漆	刷新漆	清理
1 或 3	2	3	1
2 或 4	4	6	2

例题

假设有一座陈旧的矩形木板房需要重新油漆。这项工作必须分 3 步完成：首先刮掉旧漆，然后刷上新漆，最后清除溅在窗户上的油漆。假设一共分配了 15 名工人去完成这项工作，然而工具却很有限：只有 5 把刮旧漆用的刮板，5 把刷漆用的刷子，5 把清除溅在窗户上的油漆用的小刮刀。怎样安排才能使工作进行得更有效呢？



甘特图

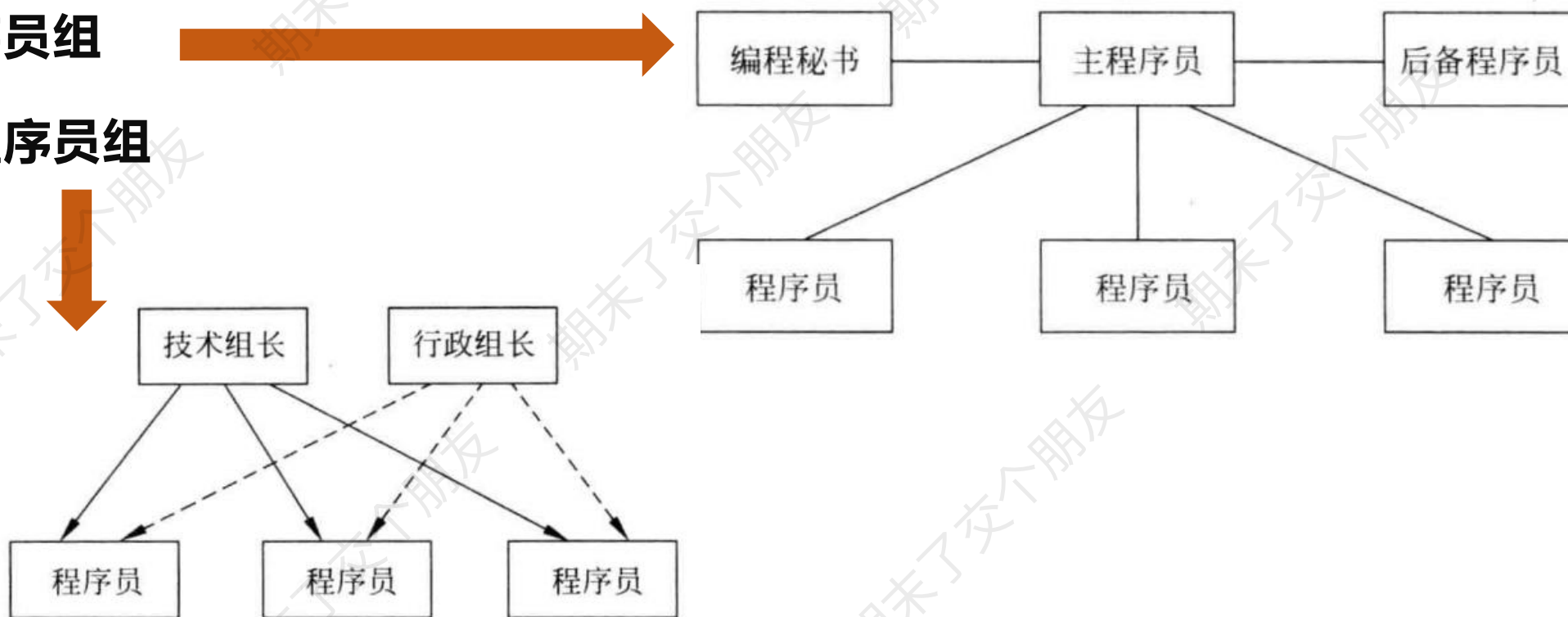


工程网络图

• **民主制程序员组**(小组成员完全平等，享有充分民主，通过协商作出技术决策)

• **主程序员组**

• **现代程序员组**



图例：



技术管理



非技术管理

(1)软件质量定义：软件与明确地和隐含地定义的需求相一致的程度。更具体地说，软件质量是软件与明确地叙述的功能和性能需求、文档中明确描述的开发标准以及任何专业开发的软件产品都应该具有的隐含特征相一致的程度

(2)软件质量因素

- 产品运行：正确性、健壮性、效率、完整性、可用性、风险
- 产品修改：可理解性、可维护性、灵活性、可测试性
- 产品转移：可移植性、可再用性、互运行性

(3)软件质量保证措施

- 复审主要用来保证在编码之前各阶段产生的文档的质量
- 基于执行的测试需要在程序编写出来之后进行,它是保证软件质量的最后一道防线
- 程序正确性证明使用数学方法严格验证程序是否与对它的说明完全一致。

- (1)**软件配置管理的定义**：软件配置管理是在软件的整个生命期内管理变化的一组活动。具体地说,这组活动用来:标识变化，控制变化，确保适当地实现了变化和向需要知道这类信息的人报告变化
- (2)**软件配置的定义**：软件过程的输出信息可以分为3类:计算机程序(源代码和可执行程序)，描述计算机程序的文档(供技术人员或用户使用)，数据(程序内包含的或在程序外的)。上述这些项组成了在软件过程中产生的全部信息，人们把它们统称为软件配置,而这些项就是软件配置项
- (3)**基线的定义**：已经通过了正式复审的规格说明或中间产品，它可以作为进一步开发的基础，并且只有通过正式的变化控制过程才能改变它。简而言之，基线就是通过了正式复审的软件配置项

(4) 软件配置管理过程

- 标识软件配置中的对象
- 版本控制
- 变化控制
- 配置审计
- 状态报告

能力成熟度模型

- **初始级**：软件过程的特征是无序的，有时甚至是混乱的。几乎没有什么过程是经过定义的，项目能否成功完全取决于开发人员的个人能力
- **可重复级**：软件机构建立了基本的项目管理过程，可追踪成本、进度、功能和质量
- **已定义级**：软件机构已经定义了完整的软件过程，软件过程已经文档化和标准化
- **已管理级**：软件机构对软件过程和软件产品都建立了定量的质量目标，所有项目的重要过程活动都是可度量的
- **优化级**：软件机构集中精力持续不断地改进软件过程

例题1 COCOMO 是 () 模型

A. 模块性成本模型

B. 质量控制模型

C. 构造性成本模型

D. 进度管理模型

例题2 软件能力成熟度模型是根据软件过程能力的递增等级，帮助组织评估和改进其软件开发能力的模型。以下 () 等级是描述一个组织已经实现了在项目和组织层面上的全面过程管理并持续改进的能力？

A. 初始级

B. 可重复级

C. 已定义级

D. 优化级